Helping Users Find New Video Games Using Scraped Data

By Harry Martin

22nd of August 2025

A project report submitted in partial fulfilment of the degree of

Master of Science in Data Science and Artificial Intelligence

School of Computer Science

Leeds Trinity University

Attestation

I understand the nature of plagiarism and other uses of unfair mean. I am aware of and have adhered to the university's policy on this.

This assignment did not user generative A.I for the purpose of completing the assignment.

Contents

Figures

Introduction

When searching for new video games, it can be hard for users to find games that they might be interested in, and it can be hard for businesses to reach new users. Using publicly available data from the internet, a database could be built alongside a user interface, such as a web application, which could help users find video games which are similar to ones they already like. This project aims to create a web application that can help users find video games they may be interested in through the use of a tagging system.

To start, a literature review will be conducted regarding this topic, specifically the reason for a tagging website, what scraping is and the reason for using scraping along with the law and security issues this project needs to acknowledge. Next, the methodology will be discussed alongside a plan of the project's scraping program, website and database. Afterwards, the development of the scraping program and the website will be documented and the data from the scraping program will be analysed with its results presented.

Overall, this project aims to answer the question: Can a tagging website be built using data scraped from the internet?

Literature Review

Why a Tagging Website?

When searching for video games, users have the options of going to a physical shop or going online and buying from a vast number of digital stores. For both options, users will be faced with an overwhelming number of video games to choose from. Castillo (2013) states that this is a problem as people ignore options on a shelf after a certain number and focus on a smaller selection instead (Specifically a selection of 3 items). They also take longer to choose. Castillo (2013) states that, when someone knows or has expertise about the items or options in a choice, they will be able to make a more confident decision than when they know little about the items or options.

However, it could also be the increasing cost of video games (Samuli, 2022) through increasingly predatory practices such as:

- Microtransactions which lock in-game content behind real world payment (Samuli, 2022) which come in a large variety of implementations such as “loot boxes” (Samuli, 2022) (Raneri et al, 2022).
- Subscription models which only allow a user to play a video game for a short period of time before needing to pay to continue accessing it (Samuli, 2022).

These practices make games more expensive than their initial price, which may, along with the number of options to choose from, make it harder for a user to decide on a video game. One more reason for difficulty in choice might also be judging a video game’s cover to represent the video game overall. Iwana et al (2016) found that, when trying to create a model to predict a book’s genre by its cover, a lot of books had ambiguous features leading to incorrect predictions. This might be the case for video games as well.

Overall, the difficulty in choice is the main reason for this project. Castillo (2013) recommends categorisation and restrictions to make choices easier and with more confidence. Assumedly, they imply that categorising helps users make more informed decisions due to knowing what the choices involve through said categorising, and assuming this is true, a website that gives users more information on video games and allows them to search for similar video games through categories (tags and genres) could help them choose new video games to play.

What is Scraping? Why Use It?

Scraping is, as described by Smith (2019) and Zhao (2017), a way to use the internet to collect public information for a specific purpose such as data analysis. It allows an easier way to collect information from a public source without needing to request it (Smith 2019).

This can be done by a human manually or a bot automatically (Zhao, 2017). An example of this given by Smith (2019) is for the indexing of a search engine by having a scraping program go from website to website to collect the information on each page, allowing each page to be searched for by comparing search requests to the contents in each page found while scraping. Smith (2019) also goes on to state the usefulness of a web scraper in building a custom dataset when one is not available.

Xiao (2021) raises the issue of privacy due to a web scraper being able to collect data posted publicly by users online without their consent. While the project will not collect any personal information from any users, data privacy will still be considered by investigating the GDPR and its UK equivalent. However, this problem applies to companies' data as well. Scrapers may also potentially violate a company's terms of service (Zhao, 2017). However, Zhao (2017) argues that it is difficult to prove the data's copyright due to "only a specific section of the data [being] legally protected", while arguing that terms of service fall into a "grey area". Nonetheless, data scraped in this project should have its source saved and, in the website, linked to in order to show that the data is from the target website and not from another source.

Overall, the main reason for using a scraping program to collect data on video games is because there is a lot of publicly available information regarding them online. Collecting this data manually would take a great amount of time. Scraping could allow for a large amount of this data to be collected in a reasonable time.

Legal and Security Issues

GDPR

Data scrapped from the target websites will not contain any personal information, however, the website will to a very minor degree. This information will be limited to a user's email address and will be used for allowing a user to create an account. Because of this, the GDPR and the UK's own data protection laws will be described here and implemented into the website.

As per the GDPR, this project needs to:

- Use data in a lawful, fair and transparent manner.
- Use data for only the purposes stated.
- Use only the necessary data needed for the stated purpose.
- Keep collected data accurate and up to date.
- Store data for only as long as it needs to be for the specified purpose.
- The data controller (me) must be held accountable for stored data.

This project also needs to follow users' rights, which the GDPR describes as the rights to:

- Be informed about data's usage.
- Have access to stored data.
- Have data be rectified.
- Have data erased upon request.
- Have data's processing be restricted.
- Have data be portable to be used in other services.
- Object to data's processing.

(GDPR, n.d.)

While similar, the UK's data protection legislation adds the following:

- Data must be handled securely to prevent against unauthorised (or illegal) processing, access, loss, destruction or damage.
- More sensitive information such as race, ethnic background, political opinions, religion, sexual orientation, etc., have stronger legal protections.

(United Kingdom, n.d.).

Computer Misuse Act 1990

Because scraping takes data from websites, the scraping program must be made in a way such that it cannot take data from an unauthorised source (CPS, 2006). Websites that would require a login or further credentials to access data should not be used. Steam is a good target because, while a user needs to login to buy games, a user does not need to sign in to access its game pages.

The Copyright, Designs and Patents Act 1988

Once a person creates something, they have copyright over their creation for a number of years depending on what kind of creation it is (for example, literary, dramatic, musical or artistic works have a copyright of 70 years, while sound recordings have 50 years instead). This copyright gives the person the right to control how their work is used and allows them to object to distortions of their work due to being identified as the author (UKCS, n.d.).

Because of this, the scraping project should, again, focus only on public data that has been put up with the author's approval. Video games viewable on public websites such as Steam implies the author's consent for their works to be viewable.

STRIDE

STRIDE is a form of security modelling for software developers that revolves around predicting potential threats during the design process of a program and attempting to mitigate them (Landuyt, 2022). This model stands for:

- Spoofing: A spoofing (faking) the identity of another user.
- Tampering: Unauthorised modification of data.
- Repudiation: A user denying they committed an action.
- Information disclosure: Unauthorised access of data.
- Denial of service: Denial of Service.
- Elevation of privilege: A user without administrative access to a system gains administrative access and gains the ability to compromise the system.

(Microsoft Learn, 2009) (Conklin et al, n.d.).

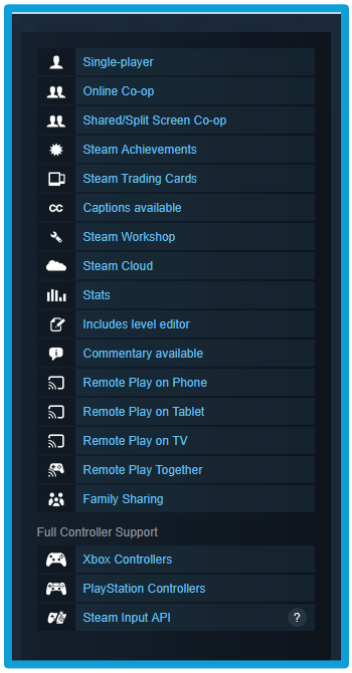
Mitigation against these threats:

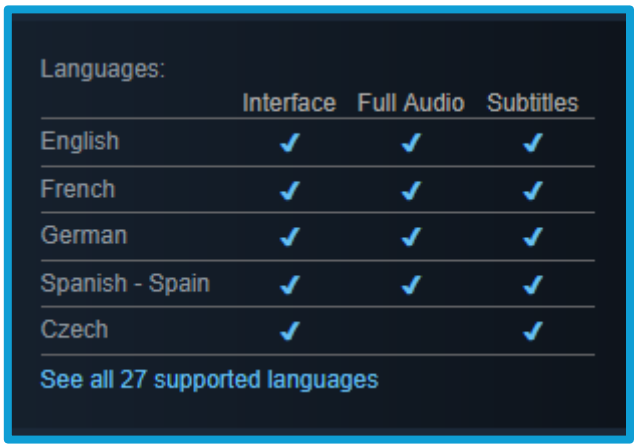
Threat	Mitigation	Meaning
Spoofing	User Authentication	Ensure the user is who they say they are through techniques such as two factor authentication.
Tampering	Data Integrity	Use encryption and hashes to protect sensitive data. Ensure sensitive data requires authorisation to modify.
Repudiation	Non-Repudiation	All actions by users should be logged by the system to prove a user's actions.
Information disclosure	Confidentiality	Only those with authorisation should be allowed to read data. Data should be encrypted.
Denial of Service	Availability	Access to the service should be throttled should users repeatedly send requests to it.
Elevation of privilege	Authorisation	Ensure users are only given the permissions needed to perform their role.

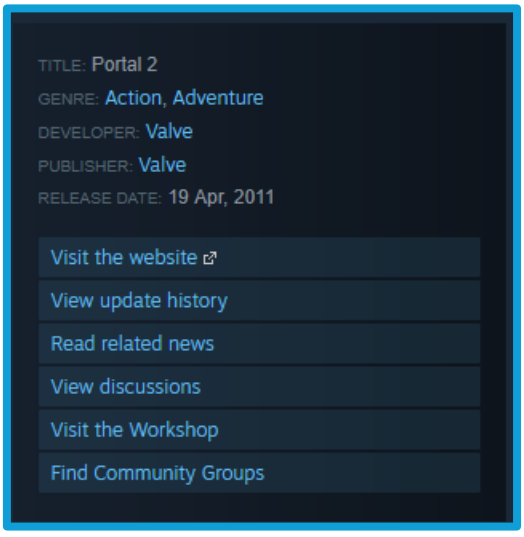
(Conklin et al, n.d.)

Ad-Hoc Data Issue

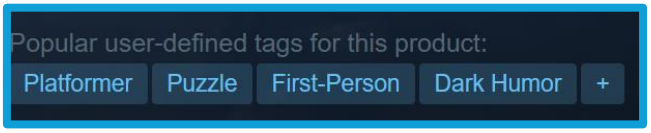
Due to the use of scraping, there is a risk of collecting data that is not entirely accurate and data that might contain biases or ethical issues. However, the main target for this program is the online video game distributor known as Steam. When publishing a video game on Steam through the “Steamworks”, the author must build their video game’s store page. The only part Steam does in the creation of this store page is reviewing it to confirm that it is working and not harmful (Steamworks, n.d.). Since this information is submitted by the author themselves, it is assumed that each page contains accurate information about each video game. Missing or erroneous information will be fixed after scraping through normalisation. Below are example images of a store page (n.d.):







One part that is not decided upon by the author is the video game tags which are described on each video game's page as "Popular user-defined tags" (Steam, n.d.):



Because of this, any data analysis performed with these tags should be considered assumptions and not factual.

Methodology

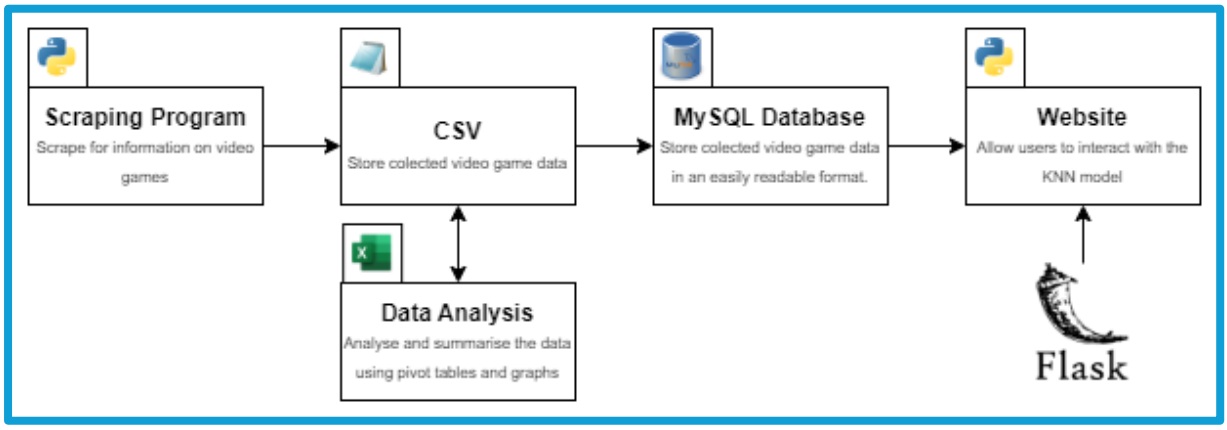


Figure 1: Development Process

The goal of this project is to create a website that allows for users to search for video games similar to ones they already like. This project will be split into the following parts:

1. **Scraping program:** This will go through the target website (Steam) and collect data about video games.
2. **CSV:** The data is then saved using the comma separated values format which will then be normalised using Excel and Python and analysed for useful information.
3. **MySQL database:** The normalised data will be loaded into a MySQL database using Python.
4. **Website:** Allows users to search for video games.

Design/Plan

Data Plan

Target Website

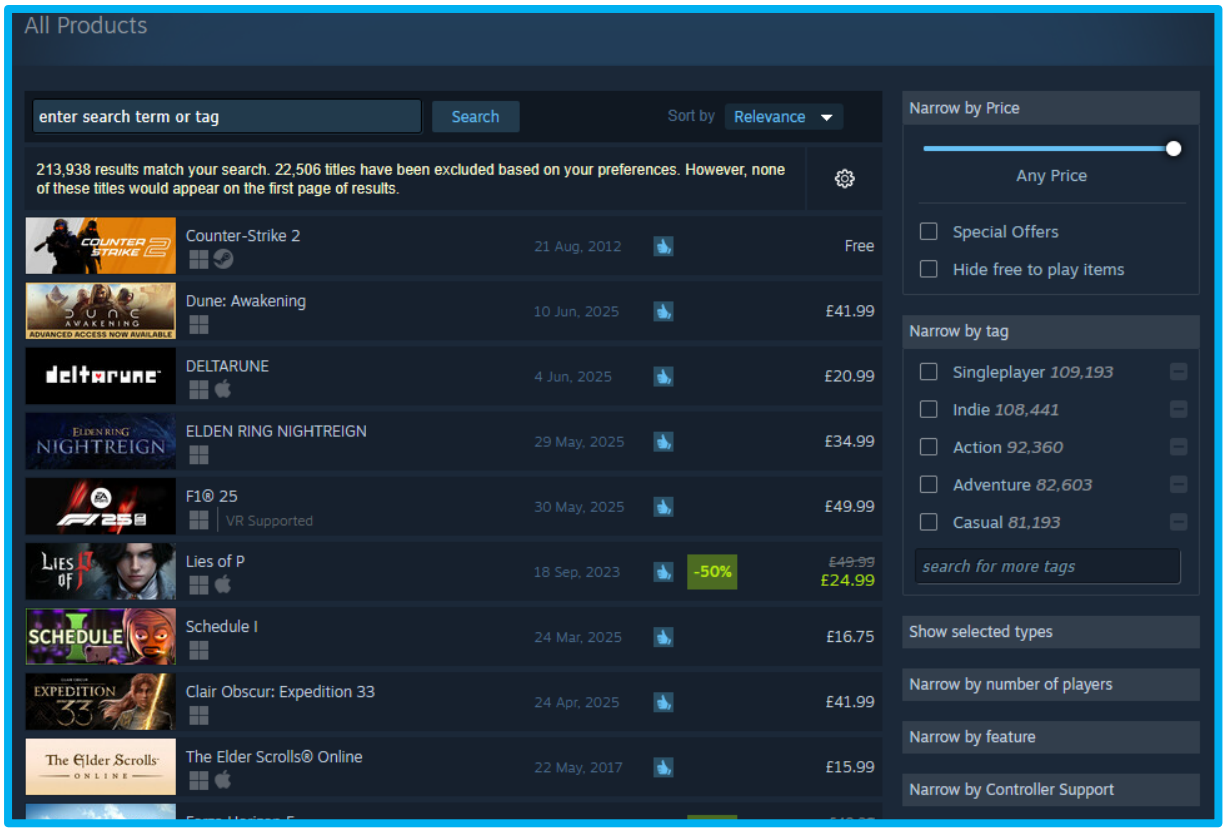


Figure 2: Steam's All Products Page

Steam has thousands of video games available to collect data from (the above screenshot shows 225, 330 games). The above page lists all of Steam's games in a useful list and gives the scraping program ways to sort the list if needed.

Scraping

For the website to work, it needs a database full of video game information and needs ways to differentiate each game. To get this data, a scraping program will used that will be programmed in Python.

To do this, several libraries could be used:

- Requests: This library allows Python to send HTTP requests to targeted web pages to receive page content such as HTML (Pypi, n.d., -a).
- Beautifulsoup: This library is a HTML parser that allows for searching of a HTML tree (Pypi, n.d., -b).
- Selenium: This library allows Python to take control of a browser (Pypi, n.d. -c).
- Schedule: This library allows for the periodic running of Python functions (Pypi, n.d. -d).

(GeeksForGeeks, 2025)

Using Steam's all games page, the following should be done for each game in the list:

1. Access the link to the game's page.
2. Obtain game information from the page's HTML.
3. Append game information to a CSV file.
4. Return to the all game's page a repeat on the next game.

When looking at the typical Steam game page. There are at least ten specific points of data that would be useful to scrape. They are:

1. Title: The title.
2. Description: Describes the game.
3. Release date: When the game was released.

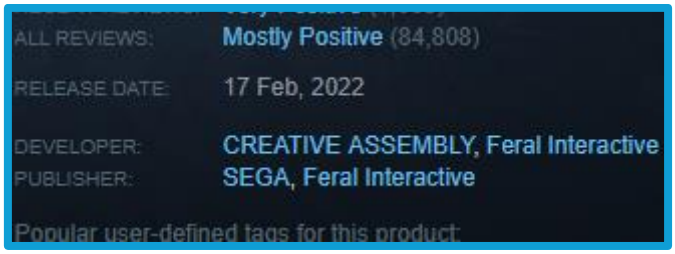


Figure 3: Steam Developers and Publishers

4. Developer(s). The people or companies that produced each game. Some developers are separated by commas.
5. Publisher(s). The company or companies that help to publish each game.
6. User defined tags. The user described content of each video game.
7. Price. The price.

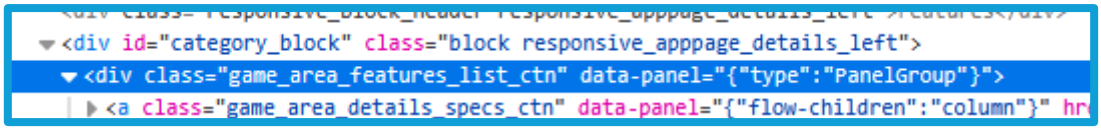


Figure 4: HTML of Game Features

8. “Features”. The features seem to describe the gameplay features that make up a game such as the game being singleplayer or multiplayer, what the game can be played on, if the game has achievements or if there are any third party agreements that need to be made to play the game. This section does not have a header and so the name of this data is taken from the HTML class it is within.
9. Languages. The game’s supported languages.
10. Genre. The game’s genres.

Saving Format (.CSV)

A comma separated values (Creativyst Software, n.d.) (Microsoft Support, n.d.) file is a filetype that stores tables in plaintext with each row and column being separated by commas (Microsoft Support, n.d.). Both Creativyst Software (n.d.) and Microsoft (n.d.) state that CSV files make it easier to share information between programs with Microsoft allowing CSV files to be imported into their Outlook software. In Python, CSV files can be opened in the same way as text files and, with the CSV library, can be treated like a dictionary of lists (Python documentation, n.d. - a).

This ease of use is the reason why this project will be using CSV files. They allow the easy saving of scraped data, which can then be loaded into Excel for normalisation. This normalised data can then be saved as another CSV file that can easily be loaded into the website using Python again.

Database (MySQL)

MySQL is an open-source database management system that allows for large amounts of data to be stored and managed (MySQL, n.d.). MySQL is relational meaning that each table has relationships defined by the programmer (MySQL, n.d.). MySQL is a structured query language (SQL) which is a standard for handling databases (MySQL, n.d.).

This project will use a MySQL database to store the data scraped from the internet. The reason for using MySQL is the ease in which Python can communicate with the database to store and receive data using the MySQL Connector Python library. This library allows Python to communicate with a MySQL database through an API which allows for the database to be connected to and allows for the execution of SQL commands (Pypi, n.d. -e).

(Continued…)

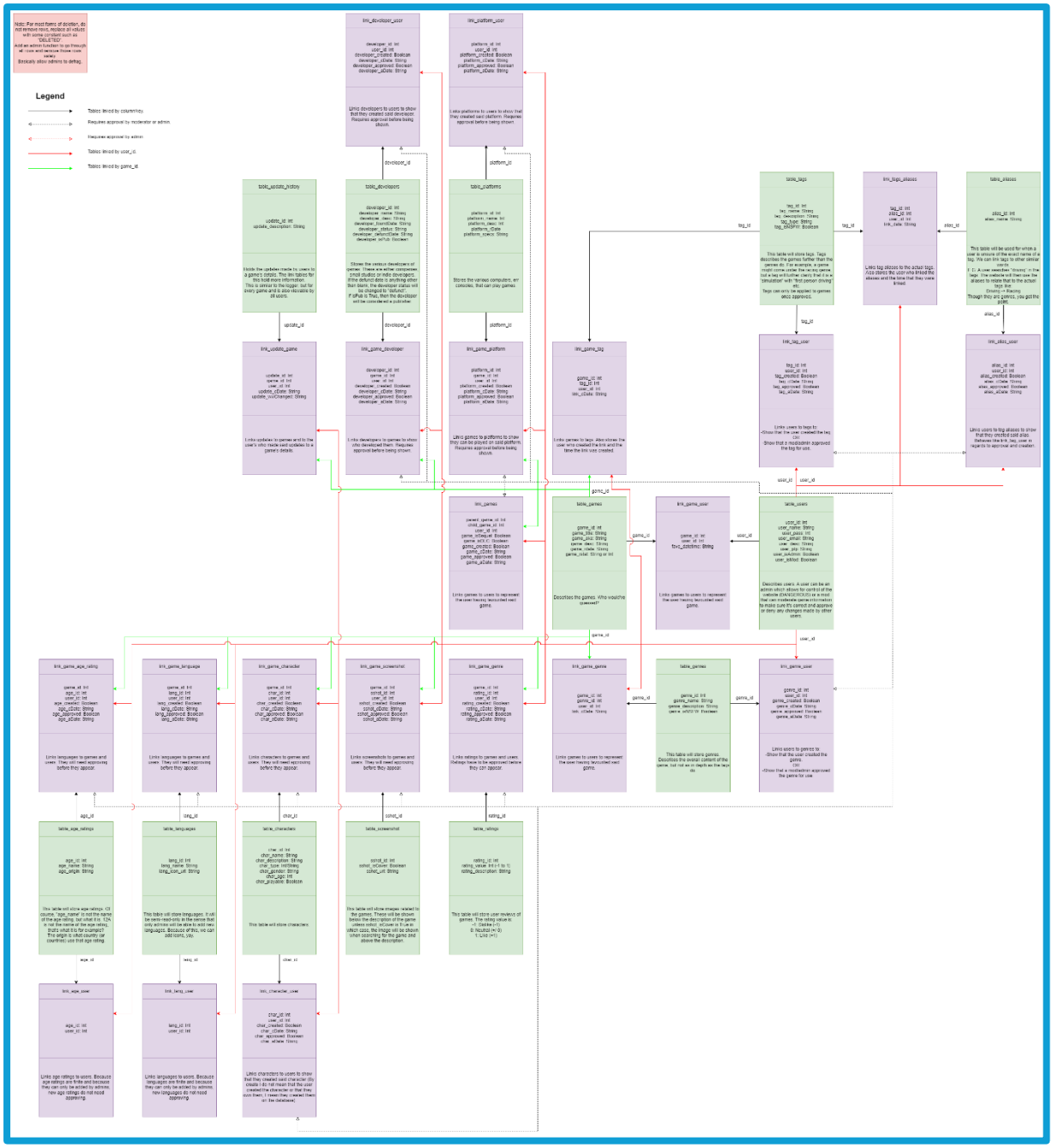


Figure 5: Database Plan

Above is a plan for the database. In it, the green boxes are regular tables that define different bits of data. The purple boxes are link tables which describe how the data in two or more tables are linked.

(Continued…)

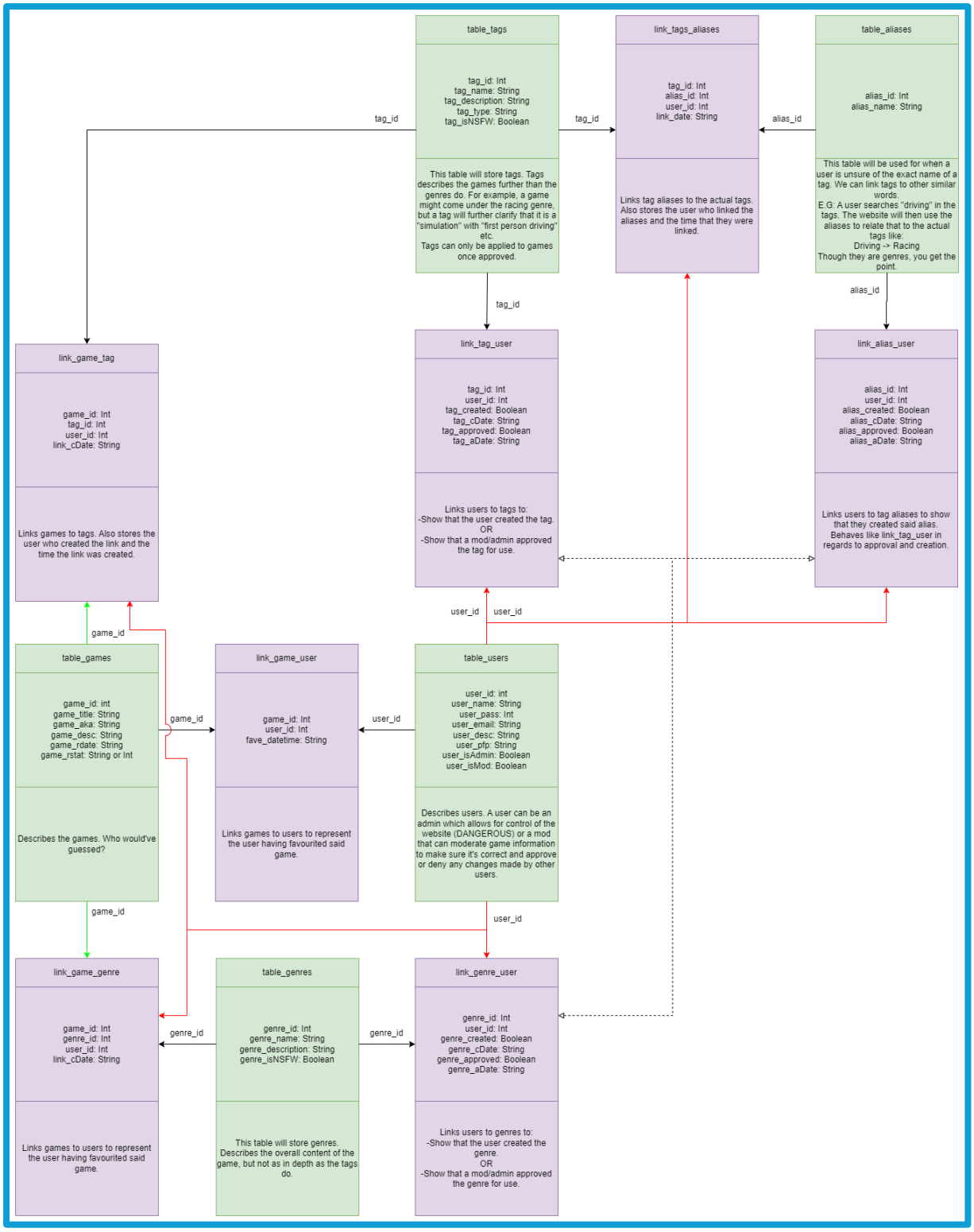


Figure 6: Database Plan (Reduced)

Here is a clearer diagram that shows only the games, users tags and tag aliases tables.

Scraping Practice

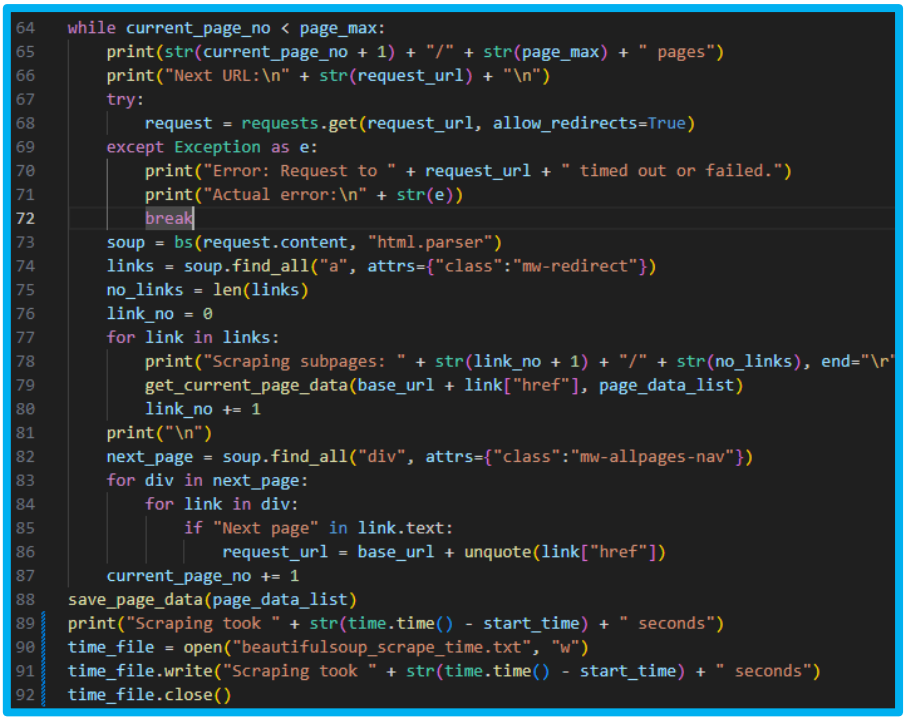
Before writing the scraping program, two practice scraping programs were made. One with the requests and Beautifulsoup libraries and the other with the selenium library. This was to learn how these libraries work, but it also helped to show which was faster. For practice, the special “All Pages” wiki page on Wikipedia was used. This page lists all of the wikis on the website as links.

The page that lists all of the wikis is split into multiple pages, and on each page is a link to a wiki page. Both programs get the HTML and search for a divider element which holds all of the links. All of the links are then looped through with their HTML being analysed for:

- Wiki Titles. This is sometimes a span element and other times it is instead a “h1” element.
- Contents links. This is an unordered list element.
- The first few paragraphs that summarise the wiki page’s topic.

After retrieving all data from the current all wikis page, the next page’s URL is found through the “Next page” link.

(Continued…)



```
while current_page_no < page_max:
    print(str(current_page_no + 1) + "/" + str(page_max) + " pages")
    print("Next URL:\n" + str(request_url) + "\n")
    try:
        request = requests.get(request_url, allow_redirects=True)
    except Exception as e:
        print("Error: Request to " + request_url + " timed out or failed.")
        print("Actual error:\n" + str(e))
        break
    soup = bs(request.content, "html.parser")
    links = soup.find_all("a", attrs={"class":"mw-redirect"})
    no_links = len(links)
    link_no = 0
    for link in links:
        print("Scraping subpages: " + str(link_no + 1) + "/" + str(no_links), end="\r")
        get_current_page_data(base_url + link["href"], page_data_list)
        link_no += 1
    print("\n")
    next_page = soup.find_all("div", attrs={"class":"mw-allpages-nav"})
    for div in next_page:
        for link in div:
            if "Next page" in link.text:
                request_url = base_url + unquote(link["href"])
    current_page_no += 1
save_page_data(page_data_list)
print("Scraping took " + str(time.time() - start_time) + " seconds")
time_file = open("beautifulsoup_scrape_time.txt", "w")
time_file.write("Scraping took " + str(time.time() - start_time) + " seconds")
time_file.close()
```

Figure 7: Beautiful Soup Scraping Main

For the requests and beautiful soup program, elements could be found with the “find all” method with beautiful soup. This returns all instances of the element requested (of which elements with specific classes or ids can be requested with the attrs attribute) which can then be looped through to handle child elements (Beautiful Soup, n.d.).

```
for list_item in contents_html: #For some reason, you must use three for loops to successfully loop through the contents list? Odd.
    for item in list_item:
        for sub_item in item:
            if sub_item != "\n":
                sub_item_text = sub_item.get_text().replace("\n", "")
                slice_counter = 0
                while slice_counter < len(sub_item_text):
                    if sub_item_text[slice_counter].isnumeric():
                        slice_counter += 1
                    else:
                        break
                sub_item_text = sub_item_text[slice_counter:]
                if re.search("\\.[0-9]+", sub_item_text) is not None:
                    sub_item_text = re.split("\\.[0-9]+", sub_item_text)
                contents_list.append(sub_item_text)
page_list.append(contents_list)
```

Figure 8: Beautiful Soup Scraping Per Page

However, from experimentation, Beautiful Soup does not allow for further searches on elements returned from a search. Allowing for another search would have made the above for loops unnecessary.

(Continued…)

```
while current_page_no < page_max:
    print(str(current_page_no + 1) + "/" + str(page_max) + " pages")
    print("Next URL:\n" + str(request_url) + "\n")
    wait.until(EC.number_of_windows_to_be(1))
    driver.switch_to.window(original_tab)
    driver.get(request_url)
    links = safe_driver_find(driver, By.CLASS_NAME, "mw-redirect")
    no_links = len(links)
    link_no = 0
    for link in links: #ToDo: Have the program get the next page link.
        wait.until(EC.number_of_windows_to_be(1))
        driver.switch_to.window(original_tab)
        print("Scraping subpage " + str(link_no + 1) + "/" + str(no_links) + " in new tab...")
        get_current_page_data(driver, link.get_attribute('href'), page_data_list)
        link_no += 1
    wait.until(EC.number_of_windows_to_be(1))
    driver.switch_to.window(original_tab)
    next_page = driver.find_elements(By.CLASS_NAME, "mw-allpages-nav")
    if len(next_page) > 0:
        for item in next_page[0].find_elements(By.TAG_NAME, "a"):
            if "Next page" in item.text:
                request_url = unquote(item.get_attribute("href"))
    else:
        print("Next page link not found, ending scraping.")
        break
    current_page_no += 1
driver.close()
print(page_data_list)
save_page_data(page_data_list)
print("Scraping took " + str(time.time() - start_time) + " seconds")
time_file = open("selenium_scrape_time.txt", "w")
time_file.write("Scraping took " + str(time.time() - start_time) + " seconds")
time_file.close()
```

Figure 9: Selenium Scraping Main

For the selenium program (using the Firefox web browser), the code is quite similar to beautiful soup. Selenium allows for the search of elements through different tag attributes rather than searching for a specific element (Selenium). This makes it easier to search for the elements without needing to know the attributes of the tags you are looking for. Also, since searching the search results is possible unlike with Beautiful Soup, looping through results to find child elements is not as necessary.

While this makes Selenium a good option for scraping, it takes a longer time to scrape than the requests library. Using the time library, both programs were timed The Beautiful Soup program took 220 seconds (about 3 minutes), while the Selenium program took 1606 seconds (about 26 minutes).

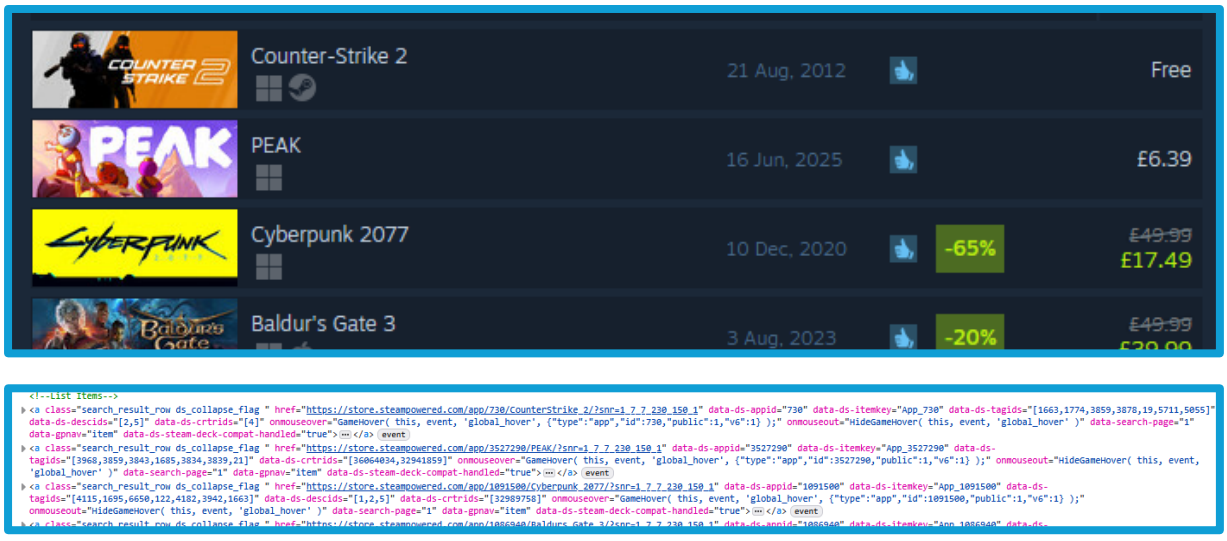
Both programs occasionally had timeout errors trying to reach Wikipedia, however, both programs worked again after restarting.

While Selenium is easier to program, requests and beautiful soup will be used instead due to being faster.

Development

Scraping Program

Main Loop and Infinite URL



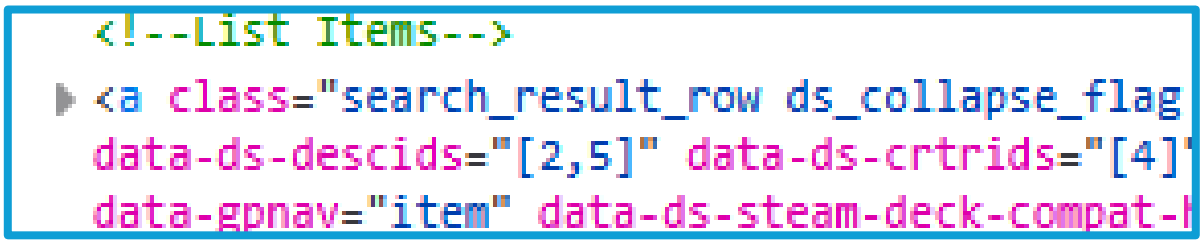


Figure 10: Per Game HTML

Before developing the scraping program, the HTML of the “all games” page on Steam was explored using the inspection tool. Each game comes in its own anchor tag that contains child tags with some of the game’s information, but more specifically, each anchor contains the URL for each game as the hypertext reference (HREF).

This was useful to learn how the HTML was laid out, but a problem arose: When displaying a large amount of data, not all of it will fit on the screen. One solution to this is infinite scrolling where more data is loaded when the user reaches the end of the page. (Interaction Design Foundation, n.d.) (Jones et al, 2016). The problem is that, once HTML is received, no additional HTML is received.

(Continued…)

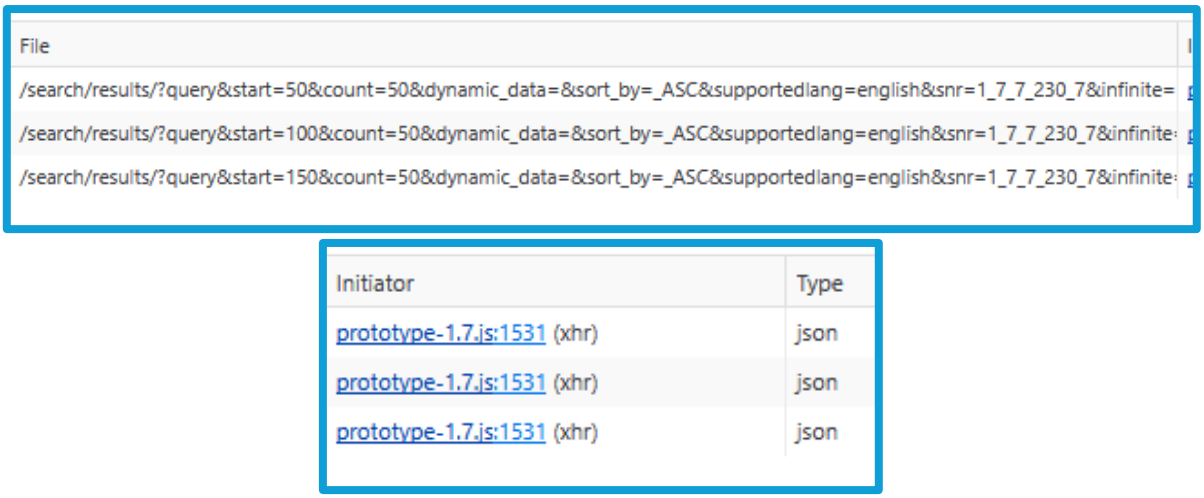


Figure 11: Steam XHR Requests

While searching for a solution to this problem, a page was found on ScrapeOps (n.d.) which suggested inspecting the network activity of the webpage for XMLHttpRequests (XHR) (MDN Web Docs) which allow for communication between JavaScript and servers to send and receive data (MDN Web Docs). Inspecting the network activity on the “all games” Steam page did yield these objects along with the URL they were requesting data from. Within these XHR URLs was a variable called “start”. This number changes what game to start at. The count variable did not work.

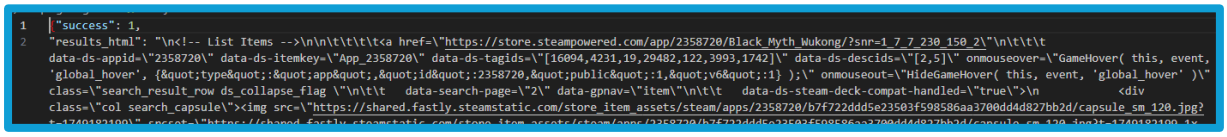


Figure 12: XHR Returned JSON

The original URL was abandoned and instead, the XHR URL was used with the count starting at zero and incrementing by 50 per fifty games returned. Though the XHR URL returned the data as a JSON file, the JSON method of the requests library was used to decode treat the data like a dictionary in Python (Requests Documentation, n.d.). From this, the HTML could be accessed with the key of “results_html”, solving infinite scrolling problem.

(Continued…)

```
while current_game_no < max_game_no:
    print("\n" + str(current_game_no) + "/" + str(max_game_no) + " games.")
    print("Next URL is:\n" + request_url + "\n")
    main_page = requests.get(request_url) #Get the next 50 games using the infinite URL.
    if main_page.status_code == 200: #If the request returned a 200 (OK) HTTP code, continue to scrape recieved game data.
        main_page.encoding = "utf-8" #Set encoding to UTF-8 to avoid encoding artefacts.
        if 'application/json' in main_page.headers.get('Content-Type'):
            main_soup = bs(main_page.json()["results_html"], "html.parser", exclude_encodings=["iso-8859-7", "iso-8859-1"])
            #Have BeautifulSoup parse the HTML from the recieved JSON file. Sometimes, BeautifulSoup would use the wrong encodin
            all_games = main_soup.find_all("a", attrs={"class": "search_result_row ds_collapse_flag"})
            #Find all of the video games in the HTML. Each one is an achor tag with the class of "search_result_row ds_collapse_
            if len(all_games) < 1: #If there are no games, that meanss the page recieved is not the correct one.
                error_check(main_soup, main_page)
            for game in all_games:
                price = game.find_all("div", attrs={"class": "discount_final_price"}) #Get the the divider that holds the game's
                if len(price) > 0: #If the price is found, save the text.
                    price = price[0].text
                else: #Some games are free and this is shown as FREE in place of the price. However, some games are free, but th
                    price = "Free"
                get_page_data(game["href"], game.find_all("span", attrs={"class": "title"})[0].text, price) #Get the game data u
                current_game_no += 1
            request_url = new_infite_url(current_game_no)
        else:
            print("Main Page request has no JSON file to read. Status code of the request was: " + str(main_page.status_code))
            if auto_retry is False:
                if input("Try again? (Y/N)").upper() in "YES" "Y":
                    print("Trying again...")
                else:
                    print("Scraping failed...")
                    import sys
                    sys.exit()
            else:
                print("Trying again...")
    else: #If the request returned any other HTTP code (usually 502 for some reason), warn the user and ask if they'd like to tr
        print("Main Page request got " + str(main_page.status_code) + " status code instead of 200.")
        if auto_retry is False:
            if input("Try again? (Y/N)").upper() in "YES" "Y":
                print("Trying again...")
            else:
                print("Scraping failed...")
                import sys
                sys.exit()
        else:
            print("Trying again...")
#With all game data collected, save it to a CSV file
```

With this solution in place, the main loop of the program (which loops until the desired number of games has been reached) requests the next fifty games. To find each game in the received HTML, it is parsed by Beautiful Soup. Then, the anchor tags mentioned at the beginning of this section are found using Beautiful Soup's find_all method (Tedboy, n.d.). In this case, the class attribute is included in the search due to the anchors having the class name of "search_result_row ds_collapse_flag".

After finding these anchor tags, all fifty games are then iterated through and their pages scraped for data.

The main body also asks to try again should the request fail.

Per Game Page Scraping

```
100     #Get user defined tags [5]
101     user_tags = []
102     tag_row = game_soup.find_all("div", attrs={"class": "glance_tags popular_tags"})
103     #Get the user tags from the 'glance tags' divider.
104     if len(tag_row) > 0:
105         for tag in tag_row[0]:
106             #For each tag, strip the text of artefacts and, if the tag is not empty a
107             tag = tag.text.strip()
108             if tag != "" and tag != "+":
109                 user_tags.append(tag)
110         current_game_data[5] = user_tags
111         #Save the user_tags list to the current_game_data list.
112     else:
113         #If the glance tags divider cannot be found, save the tags as N/A instead.
114         current_game_data[5] = na
```

Figure 13: Tag Collection

For each game page, a list is created that holds the data that will be collected called “current_game_data”. Then, for each data point that needs be collected, the program attempts to find where the data is stored in the HTML using HTML classnames and looping through the results found and adding them to their own list which is then appended to current_game_data. If no data is found, “N/A” is appended instead.

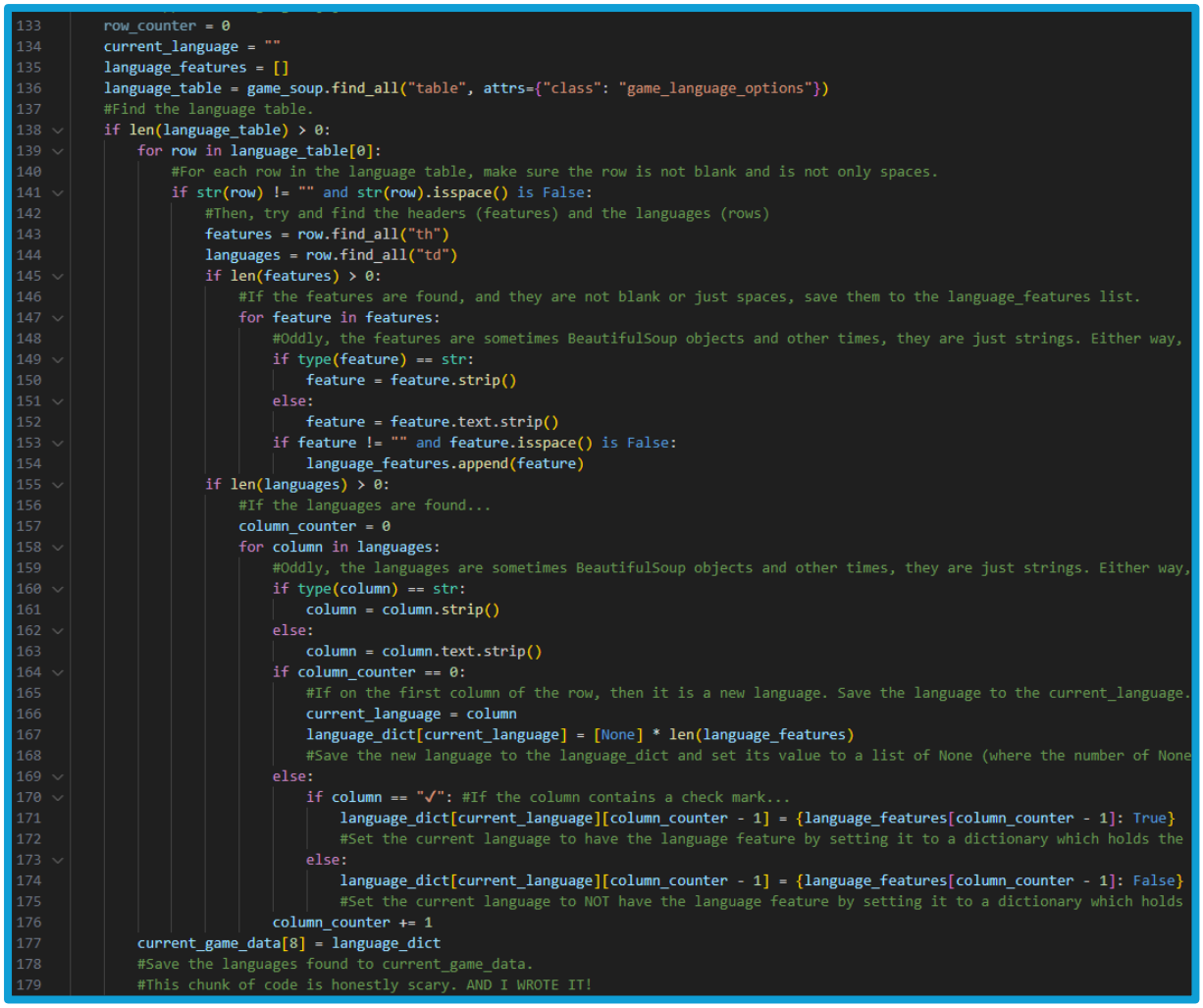
(Continued…)

```
    #Get developer and publisher [3] & [4]
    dev_row = game_soup.find_all("div", attrs={"class": "dev_row"})
    #Get the developer information from the developer divider.
    publishers = []
    developers = []
    publisher_found = False
    if len(dev_row) > 0:
        for row in dev_row:
            for item in row.find_all("div", attrs={"class": "subtitle column"}):
                if item.text.strip() == "Developer:":
                    is_developer = True
                if item.text.strip() == "Publisher:":
                    publisher_found = True
                    is_developer = False
            row_anchor = row.find_all("a") #Get the anchors in the developer divi
            for anchor in row_anchor:
                if is_developer is True:
                    if anchor.text not in developers:
                        developers.append(anchor.text)
                    current_game_data[3] = developers
                else:
                    if anchor.text not in publishers:
                        publishers.append(anchor.text)
                    current_game_data[4] = publishers
        if publisher_found is False or len(publishers) < 1:
            current_game_data[4] = na
        if len(developers) < 1:
            current_game_data[3] = na
    else:
        current_game_data[3] = na
        current_game_data[4] = na
        #If the developer divider cannot be found, save the developer and publish
```

Figure 14: Developer and Publisher Scraping

For the developers, publishers and language data points, it was not as simple. For the developers and publishers, originally, they were differentiated by checking for “developer” or “publisher” in their hypertext reference (HREF) URLs, but this led to a lot of games having blanks for developers and publishers. Instead, the “subtitle column” divider class is checked for developer or publisher. This worked, but some games do not have publishers in which case they are made N/A.

(Continued…)



```
    row_counter = 0
    current_language = ""
    language_features = []
    language_table = game_soup.find_all("table", attrs={"class": "game_language_options"})
    #Find the language table.
    if len(language_table) > 0:
        for row in language_table[0]:
            #For each row in the language table, make sure the row is not blank and is not only spaces.
            if str(row) != "" and str(row).isspace() is False:
                #Then, try and find the headers (features) and the languages (rows)
                features = row.find_all("th")
                languages = row.find_all("td")
                if len(features) > 0:
                    #If the features are found, and they are not blank or just spaces, save them to the language_features list.
                    for feature in features:
                        #Oddly, the features are sometimes BeautifulSoup objects and other times, they are just strings. Either way,
                        if type(feature) == str:
                            feature = feature.strip()
                        else:
                            feature = feature.text.strip()
                        if feature != "" and feature.isspace() is False:
                            language_features.append(feature)
                if len(languages) > 0:
                    #If the languages are found...
                    column_counter = 0
                    for column in languages:
                        #Oddly, the languages are sometimes BeautifulSoup objects and other times, they are just strings. Either way,
                        if type(column) == str:
                            column = column.strip()
                        else:
                            column = column.text.strip()
                        if column_counter == 0:
                            #If on the first column of the row, then it is a new language. Save the language to the current_language.
                            current_language = column
                            language_dict[current_language] = [None] * len(language_features)
                            #Save the new language to the language_dict and set its value to a list of None (where the number of None
                        else:
                            if column == "✓": #If the column contains a check mark...
                                language_dict[current_language][column_counter - 1] = {language_features[column_counter - 1]: True}
                                #Set the current language to have the language feature by setting it to a dictionary which holds the
                            else:
                                language_dict[current_language][column_counter - 1] = {language_features[column_counter - 1]: False}
                                #Set the current language to NOT have the language feature by setting it to a dictionary which holds
                        column_counter += 1
        current_game_data[8] = language_dict
        #Save the languages found to current_game_data.
        #This chunk of code is honestly scary. AND I WROTE IT!
```

Figure 15: Language Collection

Similarly, since the language data point is a table, collecting the supported languages within it meant having to create an algorithm to read the table.

(Continued…)

```
    #Save type (Game, software, hardware, bundles, etc.) [10] #All fou
    game_type = game_soup.find_all("div", attrs={"class": "blockbg"})
    if len(game_type) > 0:
        for item in game_type:
            item = item.text.upper()
            if "GAMES" in item:
                current_game_data[10] = "Game"
            elif "BUNDLE" in item:
                current_game_data[10] = "Bundle"
            elif "HARDWARE" in item:
                current_game_data[10] = "Hardware"
            elif "SOFTWARE" in item:
                current_game_data[10] = "Software"
            else:
                current_game_data[10] = na
    else:
        current_game_data[10] = na
    #Save game's URL
    current_game_data[11] = page_url
    #Save ALL collected game data to the game_data list.
    game_data.append(current_game_data)
```

Figure 16: Game Type

After experimenting with this scraping program, it was found that Steam also sells software, hardware and game bundles. Before the data collected is saved, the type is determined by checking the page location found at the top of the page with the class name of “blockbg” which contains the type.

After the data has been collected, the current_game_data list is appended to the “game_data” list which contains all of the game data collected.

Note: The only hardware sold on Steam is Valve’s (the company that runs Steam) own hardware. All hardware has completely different page layouts with completely different class names. Because of this, all hardware data is N/A apart from the price.

Saving to CSV

```python
def save_game_data():
    #This function saves the collected game data to a CSV file.
    #If the CSV file is open in Excel (or is failed to be written to for another reason), the user is given the choice to try sav
    while True:
        print("Saving scraped game data to CSV file...", end="")
        try:
            with open("scraped_steam_game_data.csv", "w", encoding="utf-8-sig", newline='') as csvfile:
                writer = csv.writer(csvfile)
                writer.writerow(["game_no", "game_title", "game_description", "game_release_date", "game_developer", "game_publis
                game_no = str(len(game_data))
                counter = 1
                for game in game_data:
                    print("Saving " + str(counter) + " of " + game_no + " games...")
                    game.insert(0, counter - 1)
                    writer.writerow(game)
                    counter += 1
            csvfile.close()
            print("Done")
            break
        except Exception as e:
            try:
                csvfile.close()
            except:
                pass
            print("Failed.\nFailed to save data to CSV file. Most likely the file is open in Excel.\n Actual error: " + str(e))
            if input("Try again? (Y/N): ").upper() in "YES" "Y":
                print("Trying again...")
            else:
                print("Saving failed...")
                break
```

Once all game data has been collected, it is saved to a CSV file using Python's CSV reader method as part of the CSV library (Python documentation, n.d. - a). One important part to note is that at first, the CSV files contained odd symbols within certain parts the data. It was found that this was because of incorrect encoding causing the wrong characters to be displayed (LabEx, n.d.). With research, a page of the Python Documentation (n.d) titled "Unicode HOWTO" helped to understand some of the encoding types. Through experimentation, it was found that the UFT-8-SIG encoding method prevented encoding errors.

After this, run time is saved to a file. When requesting that the program collect ten thousand games, the final time it took was 1 hour and 34 minutes (or 5651.02890920639 seconds).

Results/Findings

Data Analysis

Before analysis, the collected data was normalised. Firstly in Python using the Pandas library, to remove brackets from lists and to convert dates to numbers. Then, the data was normalised in Excel to remove N/A values.

Average Prices Per Dates

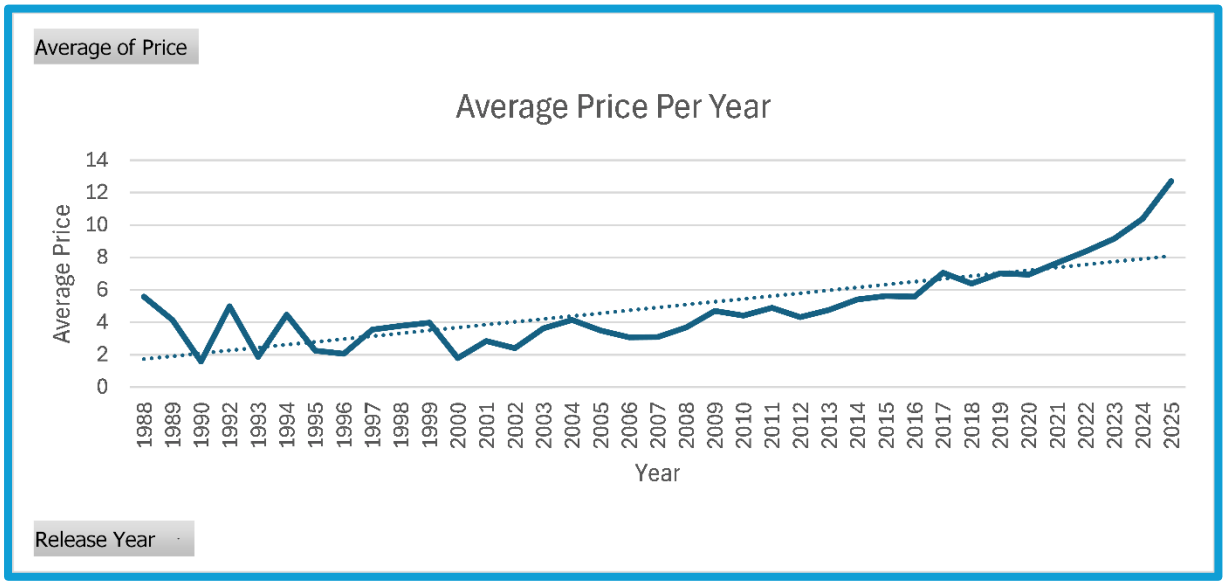


Figure 17: Average Price Per Year

To start, the prices of videogames per year and per month were investigated. For the prices of games per year, the data analysis showed a steady increase in price in games from 1988 to 2025, although this does not include the predatory practices mentioned by Samuli (2022). Interestingly, the average price rose more quickly after 2020. This could be due to the COVID-19 pandemic which caused an increase in the amount of entertainment consumed by the public (Nawaz et al, 2024) which could have caused an increase in video game prices due to labour shortages caused by COVID-19 that increased other prices, such as food (Mahmood, 2024). However, this could also be due to games from 2020 onwards not having diminished in value.

(Continued…)

Figure 18: Average Price Per Month

For average prices by month, there is no strong outlier to which to draw conclusions from. June seems to have a higher average price, but with no events in June to capitalise on, it is hard to draw a conclusion as to why.

Developers & Publishers

Figure 19: Average Game Price Per Developer

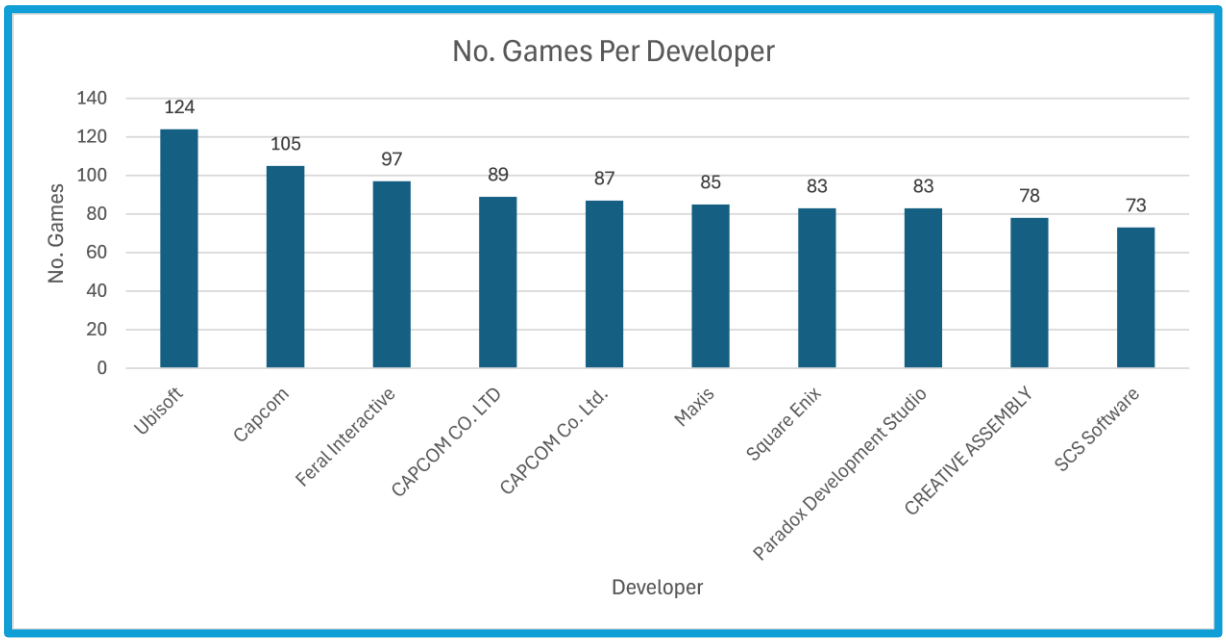


Figure 20: Number of Gamers Per Developer

Next, developers and publishers were analysed. For developers, it was found that the developer with the highest average price was “NeoBards Entertainment” with an average price of £69.99. However, (of the scraped data) the highest average price developers only have one or two games. Ubisoft for example, has 124 games which is the most of any developer and their average price is £7.14 (though this is discounting their other studios). One common link between expensive developers is that their game(s) are all very new with at least two being released in 2025 and one

being unreleased. Overall, this might be more evidence for games becoming more expensive.

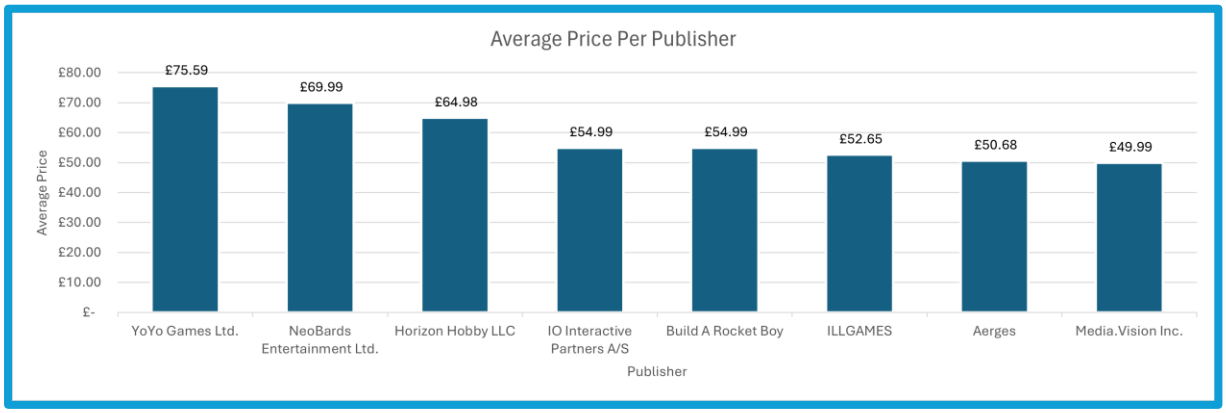


Figure 21: Average Price Per Publisher

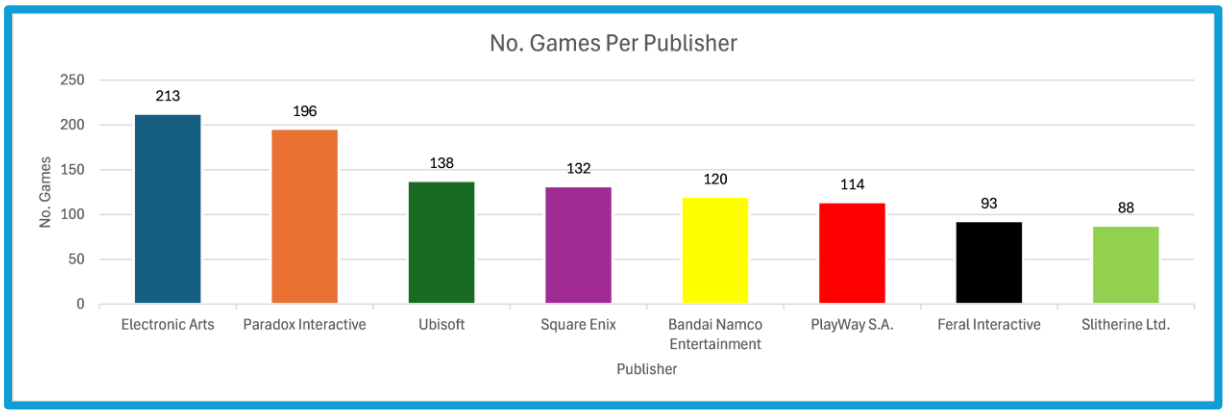


Figure 22: Number of Games Per Publisher

The most expensive publishers are roughly the same as the expensive developers which *could* suggest that developers that self-publish make their games more expensive to help with the costs of doing so. Again though, they only have one or two games.

Note: YoYo Games is featured on these graphs despite their product being software. This is because Steam strangely deems their software as a “game”, hence why it is not mentioned.

Genres & User Tags

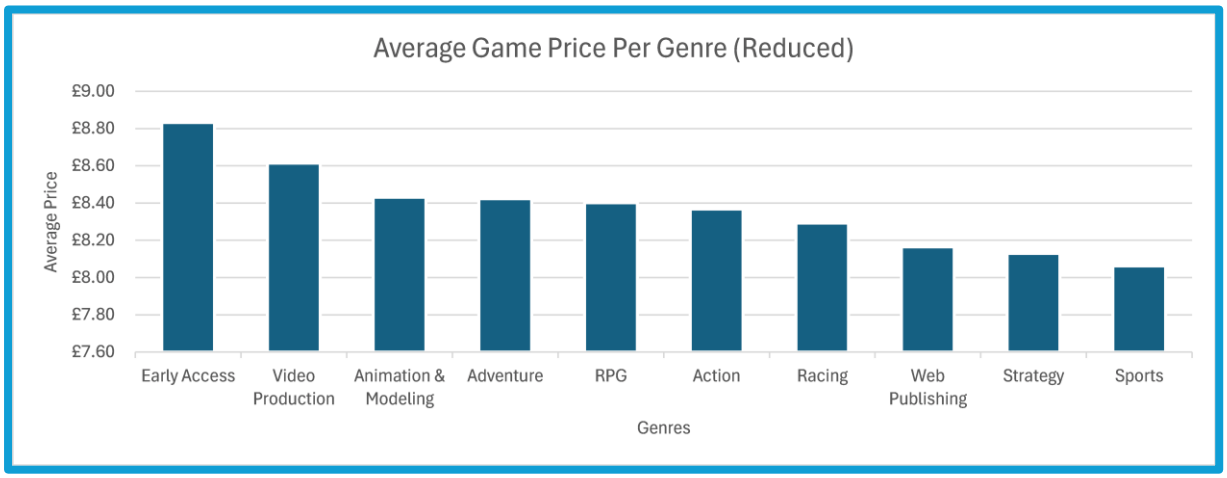


Figure 23: Average Game Price Per Genre

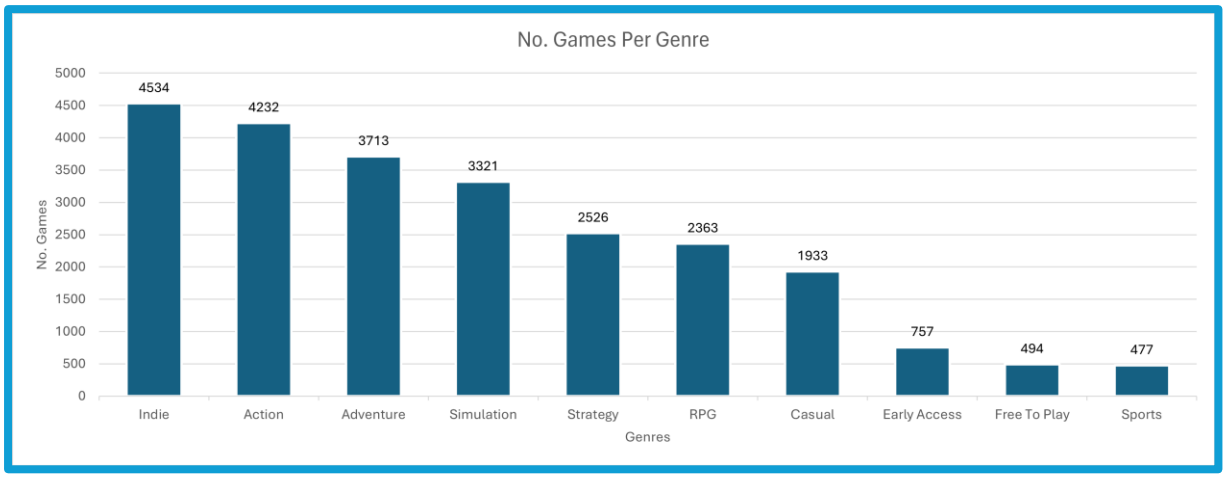


Figure 24: Number of Games Per Genre

Next, the genres and user tags were investigated. For genres, it was found that the genre with the highest average price was early access. A possible reason for this is that, unfortunately, there was a large sale when the data was scraped, this could imply that early access developers are less likely to put their games on sale. Below early access is the adventure genre which is also the third most common genre below indie and action. This could imply that adventure games are more expensive to produce and thus require higher prices to sell well. Which might be supported by notable adventure game developer, Tim Schafer, who had fans finance his next adventure instead of a publisher because “they'd laugh in my face” (Wired, 2012).

(Continued…)

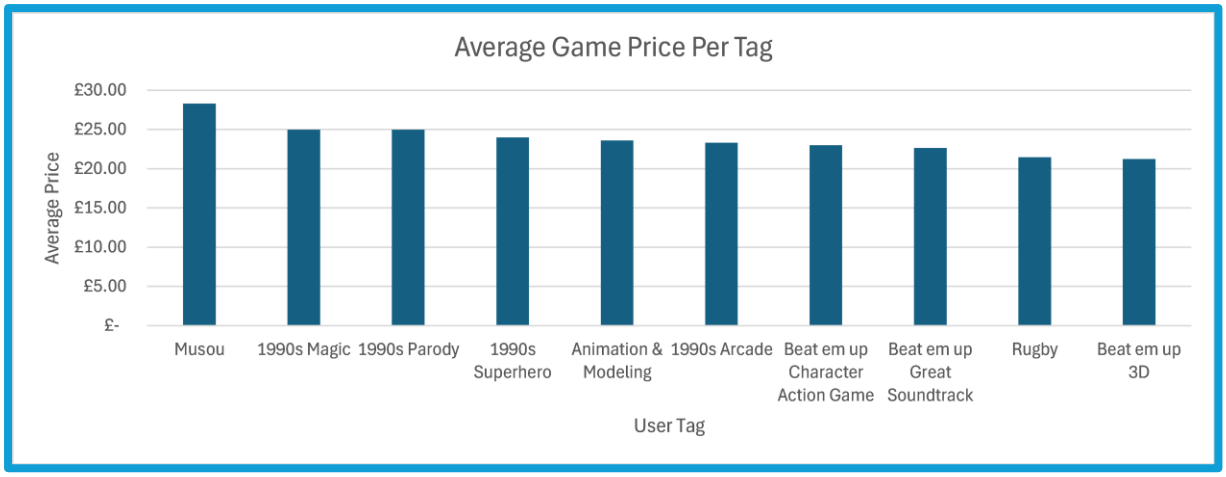


Figure 25: Average Game Price Per User Tag

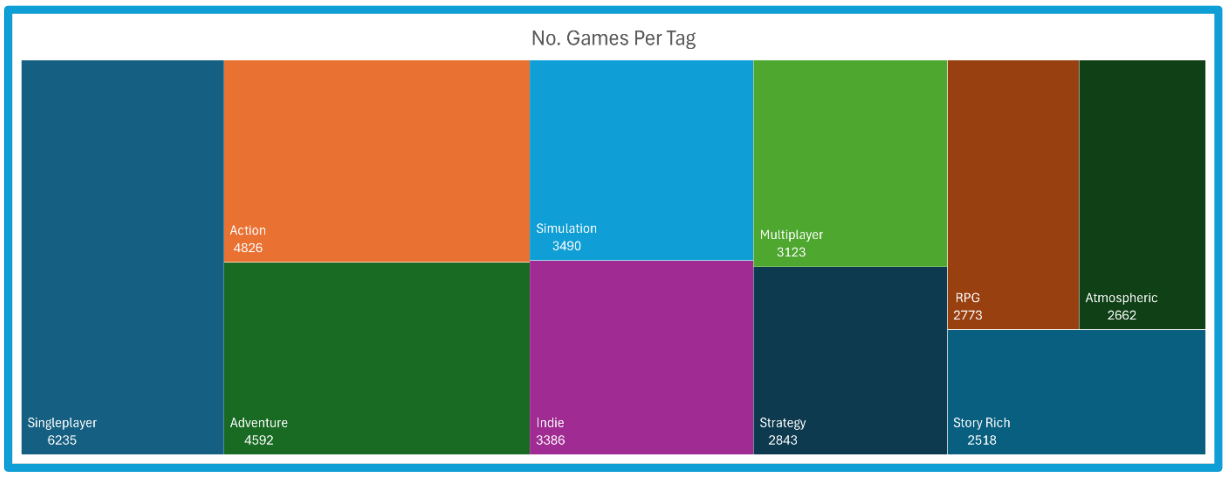


Figure 26: Number of Games Per User Tag

For user tags, the most expensive seems to be “Musou” games. Allegedly, they are “Hack ‘n’ Slash” and “Beat ‘Em Up” games similar to “Dynasty Warriors” (Cruz, 2025) which is supported by looking at the games with this tag. The main reason for this being so expensive is, again, due to the Steam sales. However, other “Beat ‘Em Up” tags are on the lower end of this graph, so it could be implied that this style of game is expensive to make, but this is unknown.

Features

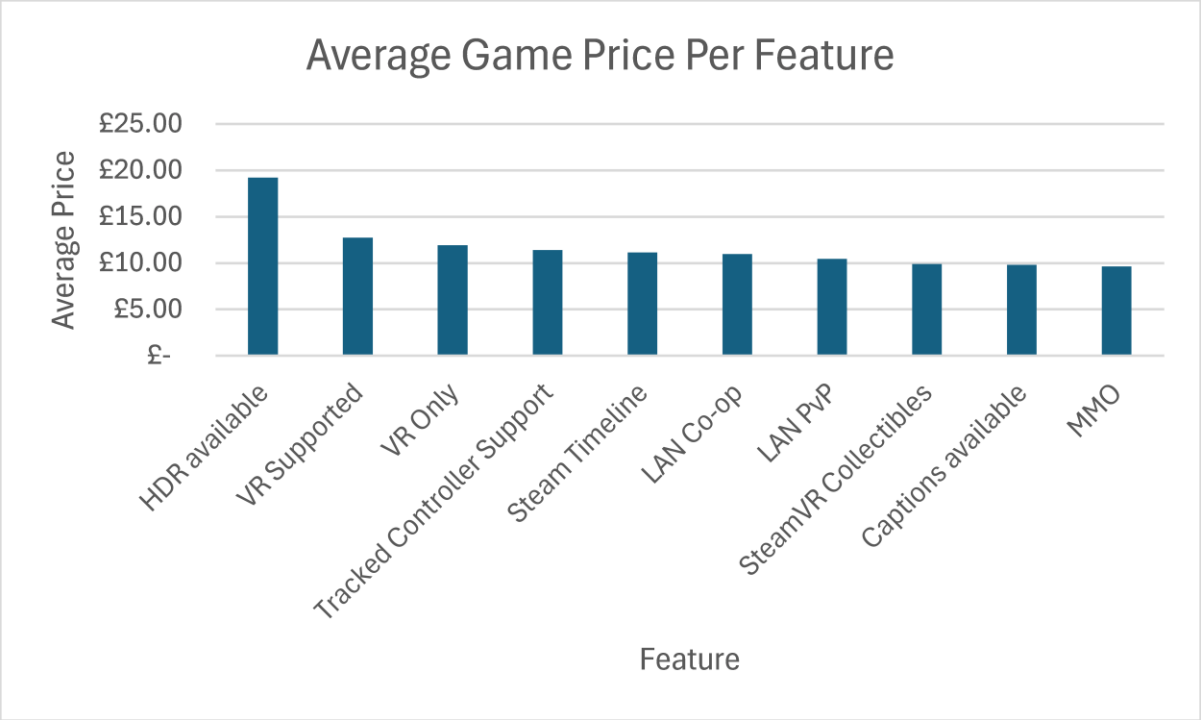


Figure 27: Average Game Price Per Feature

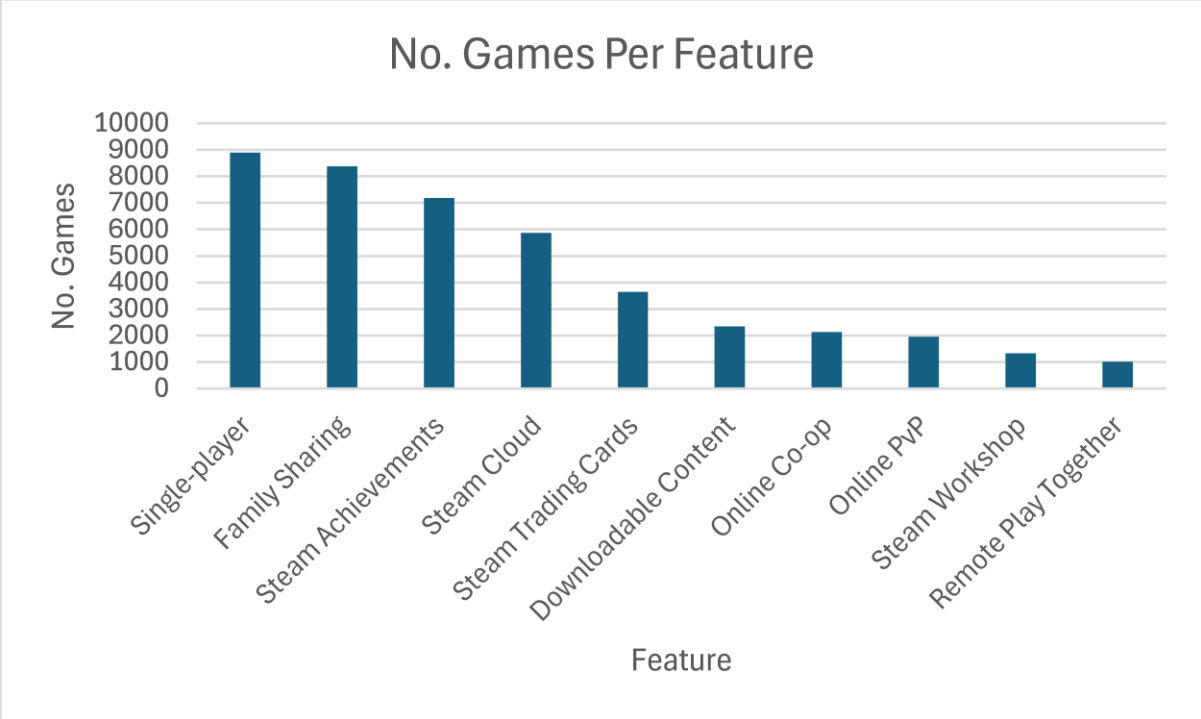


Figure 28: Number of Games Per Feature

Finally, the features were analysed. It was found that HDR available was the most expensive of all of the features, but the range of prices for the games that have this feature is quite wide going from £0.44p to £91.99. It seems the reason for this is that the expensive games that have this feature are unreleased or are very recent.

The next most expensive feature is VR which, considering how new VR is, could imply that VR is expensive to develop for.

MySQL Database

```sql
/*Describes the games themselves*/
DROP TABLE IF EXISTS table_games;
CREATE TABLE table_games(
    game_id INT NOT NULL AUTO_INCREMENT,
    game_title TEXT NOT NULL,
    game_aka TEXT,
    game_desc TEXT,
    game_rdate TEXT,
    game_rstate TEXT, /*Describes the game's release state. E.G: Early Access*/
    game_url TEXT,
    PRIMARY KEY(game_id)
);
```

Figure 29: Games Table

```sql
/*Describes the developers who make the games. Or the publishers*/
DROP TABLE IF EXISTS table_developers;
CREATE TABLE table_developers(
    developer_id INT NOT NULL AUTO_INCREMENT,
    developer_name TEXT NOT NULL,
    developer_desc TEXT,
    developer_foundDate TEXT,
    developer_status TEXT,
    developer_defunctDate TEXT,
    developer_isPub BOOLEAN DEFAULT 0,
    PRIMARY KEY(developer_id)
);
```

Figure 30: Developers Table

```sql
43  DROP TABLE IF EXISTS table_tags;
44  CREATE TABLE table_tags(
45      tag_id INT NOT NULL AUTO_INCREMENT,
46      tag_name TEXT NOT NULL,
47      tag_desc TEXT,
48      tag_type TEXT,
49      tag_isNSFW BOOLEAN DEFAULT 0,
50      PRIMARY KEY(tag_id)
51  );
```

Figure 31: Tags Table

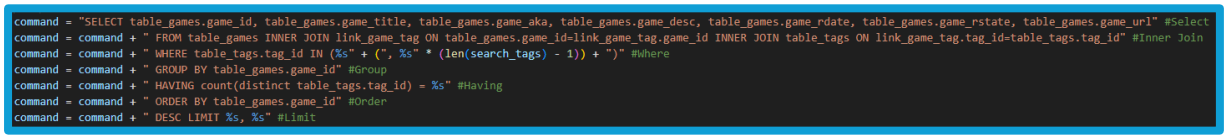
All of the tables were implemented and can be viewed when exploring the database, however, only four of them are used by the website with those being the above three and the users table. The database works well and is able to store game data in the way that was planned, though there were some modifications to the database different from the design such as adding the ability for inputted data be denied.

(Continued…)

```
def game_get_single(game_id=0):
    try:
        database = mysql.connector.connect(**get_db_config(deployed))
        cursor = database.cursor()
        cursor.execute("SELECT * FROM table_games WHERE game_id = %s", (game_id,))
        fetch = cursor.fetchall()
        cursor.close()
        database.close()
        if len(fetch) > 0:
            fetch = fetch[0]
            game_data = {
                "game_id": fetch[0],
                "game_title": fetch[1].replace("_", " ").title(),
                "game_aka": fetch[2],
                "game_desc": fetch[3],
                "game_rdate": fetch[4],
                "game_rstate": fetch[5],
                "game_url": fetch[6]
            }
            for item in game_data:
                fprint(type(game_data[item]))
            return game_data
        else:
            return None
    except:
        return None
```

Figure 32: Example Database Function

The database was communicated with through four files (named database handlers) which handled users, administrator commands, table links and more general requests such as retrieving game information. All of these files contain functions that follow the pattern seen in the above image, that is: They connect to the database, execute SQL commands, close the connection and then return any data.



```
command = "SELECT table_games.game_id, table_games.game_title, table_games.game_aka, table_games.game_desc, table_games.game_rdate, table_games.game_rstate, table_games.game_url" #Select
command = command + " FROM table_games INNER JOIN link_game_tag ON table_games.game_id=link_game_tag.game_id INNER JOIN table_tags ON link_game_tag.tag_id=table_tags.tag_id" #Inner Join
command = command + " WHERE table_tags.tag_id IN (%s" + (", %s" * (len(search_tags) - 1)) + ")" #Where
command = command + " GROUP BY table_games.game_id" #Group
command = command + " HAVING count(distinct table_tags.tag_id) = %s" #Having
command = command + " ORDER BY table_games.game_id" #Order
command = command + " DESC LIMIT %s, %s" #Limit
```

Figure 33: Search Games With Tag Command (Code)

```
SELECT table_games.game_id, table_games.game_title, table_games.game_aka,
table_games.game_desc, table_games.game_rdate, table_games.game_rstate,
table_games.game_url FROM table_games
INNER JOIN link_game_tag ON table_games.game_id=link_game_tag.game_id INNER
JOIN table_tags ON link_game_tag.tag_id=table_tags.tag_id
WHERE table_tags.tag_id IN (1, 4, 3, 37)
GROUP BY table_games.game_id
HAVING count(distinct table_tags.tag_id) = 4
ORDER BY table_games.game_id
DESC LIMIT 0, 10
```

Figure 34: Search Games With Tag Command (Example)

When getting all game data, a simple SQL command is run that gets all of the approved games and limits the results to 10 items per page. However, if the user searches for games using tags, a more complex SQL command is used instead. This command was hard to implement to the point that it was split into multiple parts (as shown in figure 33) for clarity. It works by finding the games that are linked to the tag

IDs of the tags the user inputted using the “IN” command in the WHERE section. It then selects the games that have ALL of those tags using the HAVING command. An example of this command with actual values is shown in figure 34 where the user searched for games that have the following tags: 1: Action, 4: Shooter, 3: FPS and 37: 3D.

Before this was added, the code instead got all of the game’s that had the first provided tag, and then Python would loop through them to find games with all of the tags. This proved to be far too slow for a website however as it took about 5 to 10 seconds for a page to load. The MySQL command loads almost instantly.

```
def read_scraped_data(user_id):
    database = mysql.connector.connect(**get_db_config(deployed))
    cursor = database.cursor()
    with open(scraped_file_dir, "r", encoding="utf-8-sig") as scraped_data:
        reader = csv.reader(scraped_data)
        data = list(reader)
        row_length = len(data)
        if limit is not None:
            if row_length > limit:
                row_length = limit
        row_counter = 0
        game_dict = {}
        for row in data:
            if row_counter > 0:
                game_dict = {
                    "game_title": row[0],
                    "game_description": row[1],
                    "game_developers": convert_to_list(row[2]),
                    "game_publishers": convert_to_list(row[3]),
                    "game_user_tags": convert_to_list(row[4]),
                    "game_features": convert_to_list(row[6]),
                    "game_languages": convert_to_list(row[7]),
                    "game_genres": convert_to_list(row[8]),
                    "game_url": row[9],
                    "game_release_year": row[10],
                    "game_release_month": row[11],
                    "game_release_day": row[12],
                    "link_user_id": user_id
                }#Price is not included here because this website is a database of games, not a storefront.
                admin_add_scraped_data(game_dict, user_id, database, cursor)
            row_counter += 1
            print(str(row_counter + 1) + "/" + str(row_length) + " rows loaded from scraped_steam_gamescraped_steam_game_data.csv", flush=True, end="\r")
            if row_counter >= row_length:
                break
        print(str(row_counter + 1) + "/" + str(row_length) + " rows loaded from scraped_steam_gamescraped_steam_game_data.csv\nLoading finished.", flush=True)
    scraped_data.close()
    cursor.close()
    database.close()
```

Figure 35: CSV to Database Loader

To search for games however, there needs to be data in the database. With the data scraping of Steam having been performed, the data is loaded from the created CSV file using the CSV reader method of the CSV library as shown above. The reader creates dictionaries out of the columns on each row which are then passed to the “admin_add_scraped_data” function where the data is added to the database and is linked to the admin user who started the loading of the CSV file. For 100 games, this process roughly takes 30 seconds and for the full dataset, it roughly takes 30 minutes. However, this process is slower on other computers.

(Continued…)

```
cursor.execute("INSERT INTO table_games VALUES(%S, %s, %s, %s, %s, %s, %s)",
(game_dict["game_id"], game_dict["game_title"], None,
game_dict["game_description"], release_date, released,
game_dict["game_url"],))
```

Figure 36: Example Game Insert Command

```
cursor.execute("INSERT INTO table_games (game_title, game_desc) VALUES(%s,
%s)", (game_dict["game_title"], game_dict["game_description"],))
```

Figure 37: Example Game Insert Command (Simplified)

During the making of this code, it was discovered that the columns to be inserted could be declared so that only those values would need to be provided with the undeclared columns using their default values instead. This allowed for SQL insertion commands to be simpler as shown above. Figure 36 is without declaring the columns where ALL values must be provided, while figure 37 is declares the columns and is shorter as a result.

Website

Alongside the database was the website which allows for users to: Signup and login along with viewing games, tags, developers and publishers and adding new games, tags, developers and publishers as long as they are logged in. It also allows them to, if they are moderators, approve new games, tags, developers and publishers so that users can see them. Users can also change the tags that a game has. This section will go over screenshots of the website and explain them.

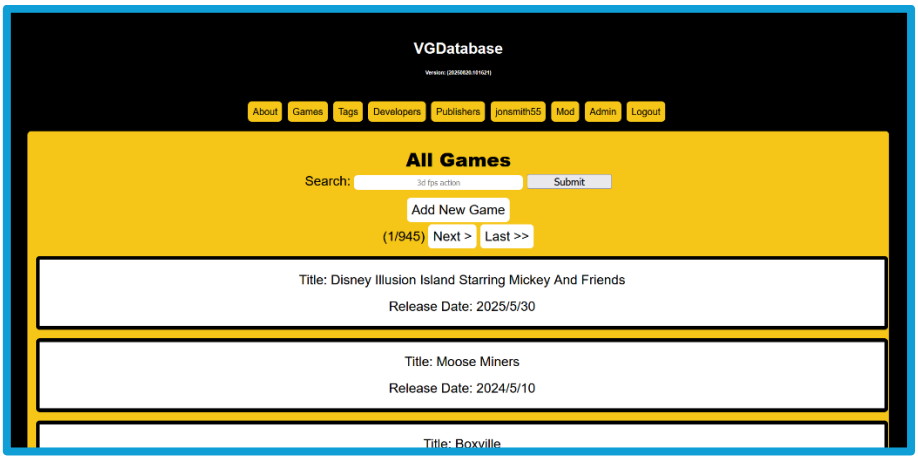


Figure 38: All Games Page

Figure 39: All Games Page With Tag Search

Firstly, there is the all games page shown in figure 38. This page lists all of the *approved* games in groups of 10 per page with a pagination system to navigate said pages. The number of pages reduces when the user searches with tags as shown in figure 39. From this page, users can click on each game and be taken to that game's page.

(Continued…)

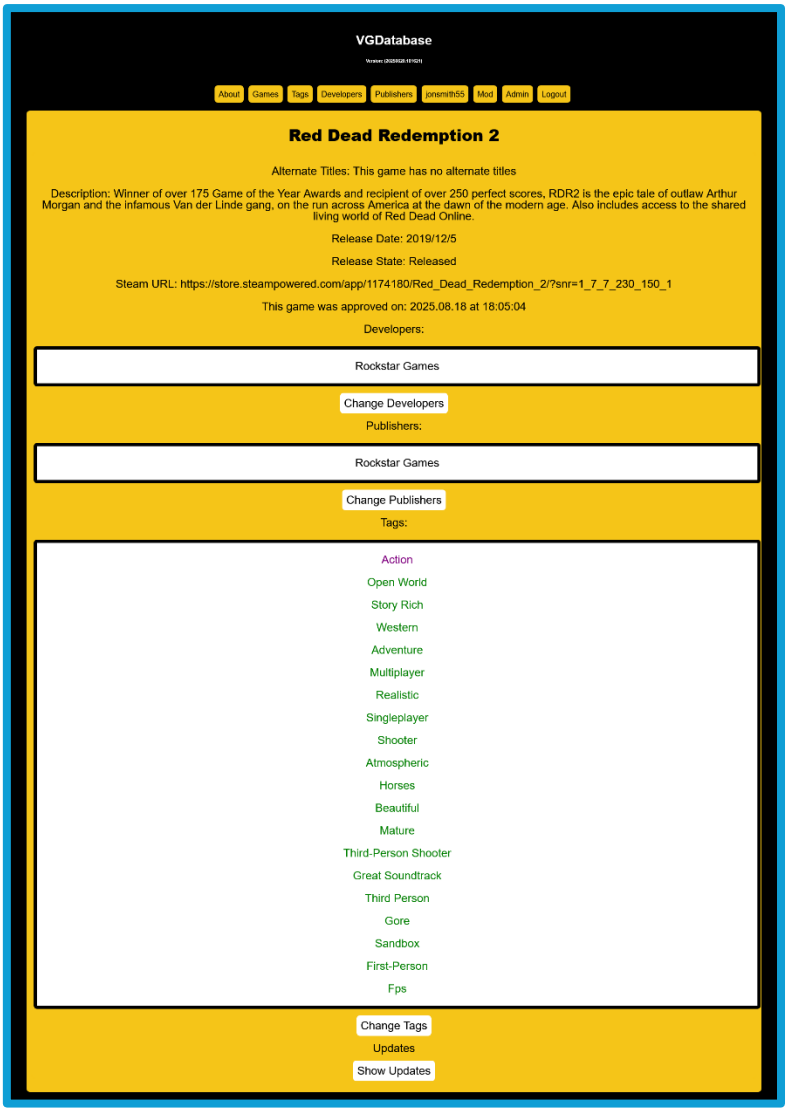


Figure 40: Individual Game Page

Each individual game page shows the game's title, description, release date, release state and Steam URL along with the game's developers, publishers, tags and the time when the game was approved. From here, users can change the developers, publishers and tags. They can also click any of the developers, publishers or tags to be taken to their page.

(Continued…)

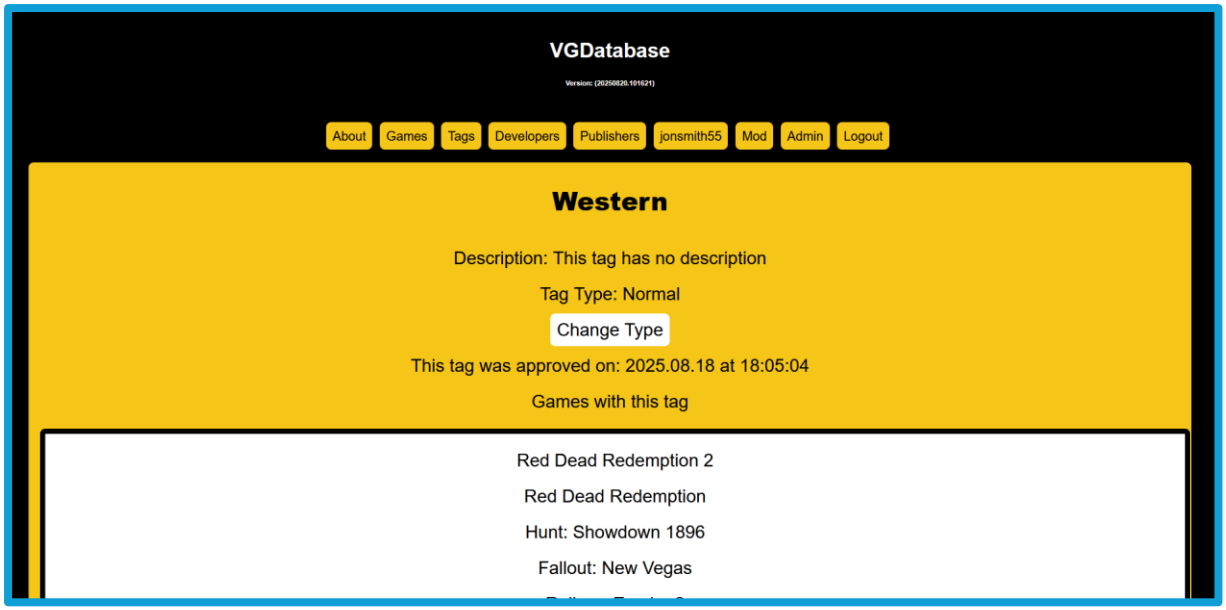


Figure 41:Individual Tag Page

Similarly, the individual developer, publisher and tag pages show their name, description and when they were approved, however, they also show the games they are linked to as shown in figure 41.

(Continued…)

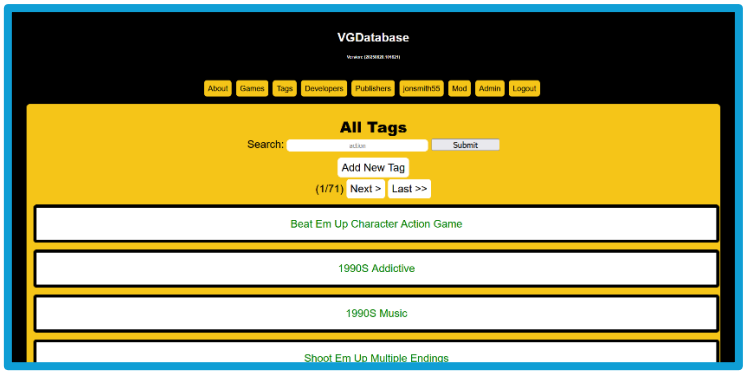


Figure 42: All Tags Page

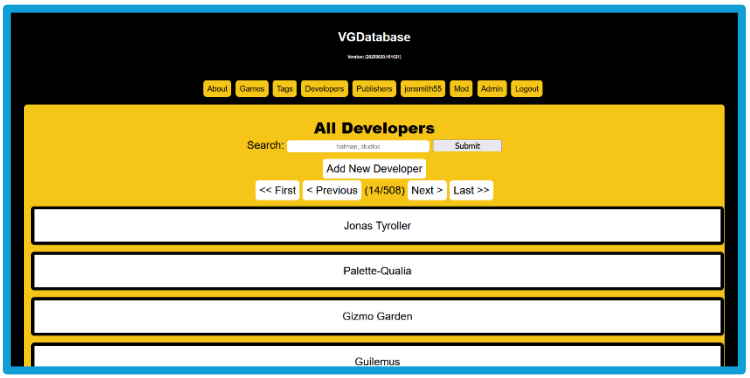


Figure 43: All Developers Page

Figure 44: All Publishers Page

There are also pages that show all of the developers, publishers and tags which are similar to the all games page. However, their search bars search for similar entries in the database to what the user typed, rather than the exact values.

(Continued…)

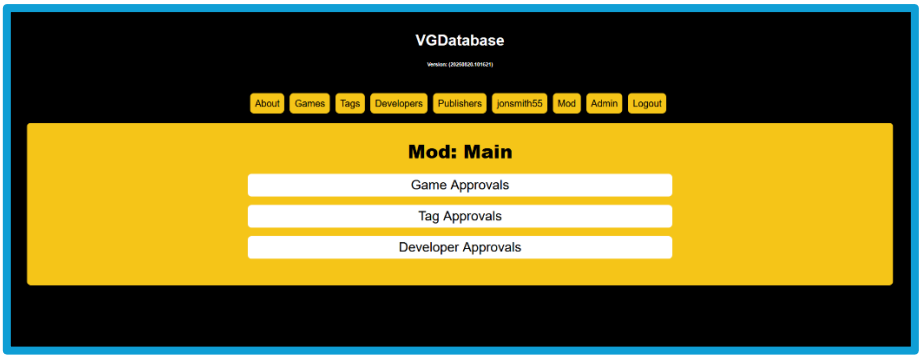


Figure 45: Mod Page

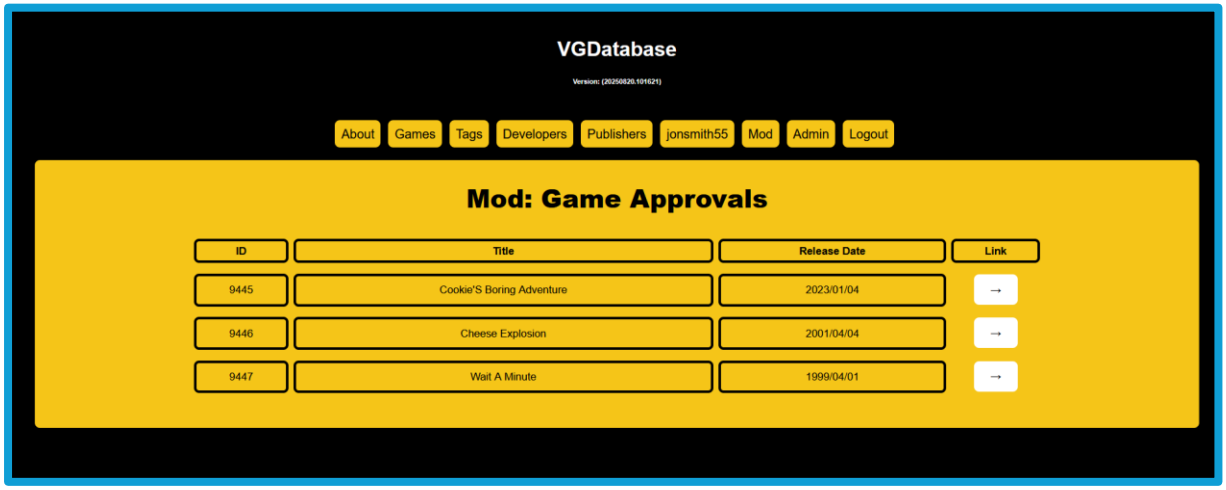


Figure 46: Mod Game Approval Page

Lastly, there are the moderator pages which allow moderators to view unapproved games, tags and developers (which are referred to as “devpubs” in some sections of the website’s code) From here, the moderators can then be taken to the unapproved item’s page which will look similar to the below image:

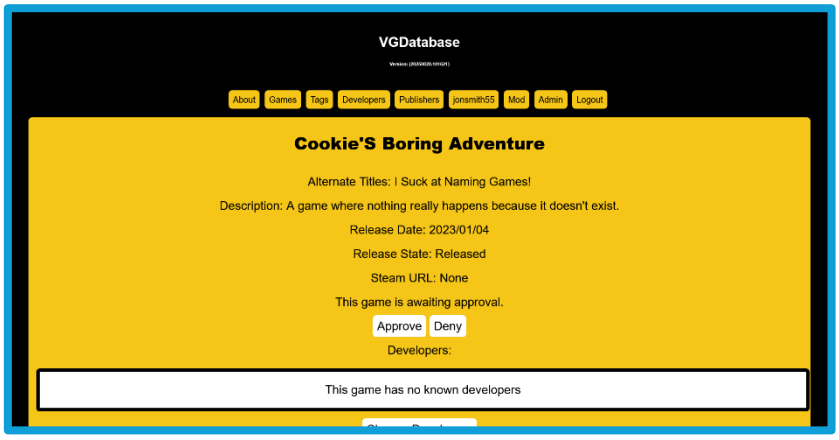


Figure 47: Unapproved Game

(Continued…)

On the item’s page, the moderator then has the ability to approve it so that other users can see it, or to deny it along with a reason for denial. The original intent here was to allow users to see denied items along with the reasons as to why they were denied, but this was not finished in time.

Overall, the website and database work well together. Searching for games is quick and easy, and other tasks (such as adding a new game) are even quicker. It also successfully meets the goal of being able to search for games by tags. However, while it works well, it could do with being refined in certain ways. For example, visually, the website is very poor and basic. Regardless, the current iteration of the website makes a really good foundation from which to create an even better video game database website.

For future versions, the website should be visually cleaned up along with implementing the features that were not done due to the current time constraints. Examples of those include games have their box art, users being able to favourite games, etc. Some features, such as users modifying their accounts, should be made to be less awkward. Lastly, the search feature should be changed to allow users to have more options when searching such as being able to search by date, search for games by developers and publishers instead of just tags, among other options.

Conclusion

Overall, the scraping program, website and database successfully followed the objectives set in the methodology. The scraping program is able to gather a large enough dataset in a reasonable time. The dataset was normalised and able to be loaded into the database. And the website allowed for users to successfully search for games using tags as well as allowing them to add new data. Surprisingly, the scraping program was easier and quicker to implement than the website. The former took around three weeks while the latter took roughly one and a half months.

Regarding the question posed in the introduction: A tagging website can be successfully built using data scraped from another website. Arguably, the use of scraped data made building the website faster due to having data to test implementations with.

Moving forward, when continuing the project (which is intended to be continued), all of the code should be cleaned to be more streamlined and clearer alongside new features. The website's design should also be improved as it's current CSS is very basic and potentially hard for users to follow.

References

Final word count: 5998/6000 (+10%: 6600)

- Beautiful Soup. (n.d.). Beautiful Soup Documentation. Beautiful Soup. https://beautiful-soup-4.readthedocs.io/en/latest/
- Castillo, M. (2013, Jun). Overwhelmed by Choices. American Journal of Neuroradiology : AJNR, 34(6), 1111–1112. https://doi.org/10.3174/ajnr.A3274
- Conklin, L., Drake, V., Strittmatter, Sven., Braiterman, Z., & Shostack, A. (n.d.). Threat Modeling Process. OWASP. https://owasp.org/www-community/Threat_Modeling_Process
- CPS. (2023, August 3rd). Computer Misuse Act. The Code for Crown Prosecutors. https://www.cps.gov.uk/legal-guidance/computer-misuse-act
- CPS. (2024, March 11th, b). Fraud Act 2006. The Code for Crown Prosecutors. https://www.cps.gov.uk/legal-guidance/fraud-act-2006
- GDPR. (n.d.). What is GDPR, the EU's new data protection law? GDPR. https://gdpr.eu/what-is-gdpr/
- Creativyst Software. (n.d.). How To: The Comma Separated Value (CSV) File Format. Creativyst Software. https://www.creativyst.com/Doc/Articles/CSV/CSV01.shtml
- Cruz, C. (2025, January 17th). How 'Dynasty Warriors' Created One of Gaming's Best Action Subgenres. RollingStone. https://www.rollingstone.com/culture/rs-gaming/musou-dynasty-warriors-explained-1235238935/
- GeeksForGeeks. (2025). Python Web Scraping Tutorial. GeeksForGeeks. https://www.geeksforgeeks.org/python-web-scraping-tutorial/
- Interaction Design Foundation. (n.d.). Infinite Scrolling. Interaction Design Foundation. https://www.interaction-design.org/literature/topics/infinite-scrolling
- Iwanna, B., Rizvi, S., Ahmed, S., Dengel, A., & Uchida, S. Judging a Book By its Cover. Cornell University. https://doi.org/10.48550/arXiv.1610.09204
- Jones, K., & Martin, L. (2016). Adapting infinite-scroll with user experience in mind. DiVA. https://www.diva-portal.org/smash/record.jsf?pid=diva2%3A972535&dswid=1795
- LabEx. (n.d.). How to manage CSV file encoding. LabEx. https://labex.io/tutorials/python-how-to-manage-csv-file-encoding-418947
- Landuyt, D., & Joosen, W. (2022). A descriptive study of assumptions in STRIDE security threat modelling. Software and Systems Modelling, 21(6), 2311-2328. https://doi.org/10.1007/s10270-021-00941-7
- Mahmood, H., Furqan, M., Meraj, G., & Hassan, M. (2024, April 23rd). The effects of COVID-19 on agriculture supply chain, food security, and environment: a review. Environmental Science. https://peerj.com/articles/17281/

- MDN Web Docs. (n.d.). XMLHttpRequest. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/XMLHttpRequest
- Microsoft Learn. (2009). The STRIDE Threat Model. Microsoft Learn. https://learn.microsoft.com/en-us/previous-versions/commerce-server/ee823878(v=cs.20)?redirectedfrom=MSDN
- Microsoft Support. (n.d.). Create or edit .csv files to import into Outlook. Microsoft Support. https://support.microsoft.com/en-gb/office/create-or-edit-csv-files-to-import-into-outlook-4518d70d-8fe9-46ad-94fa-1494247193c7
- MySQL. (n.d.). 1.2.1 What is MySQL? MySQL Documentation. https://dev.mysql.com/doc/refman/8.0/en/what-is-mysql.html
- Nawaz, S., Bhowmik, J., Linden, T., & Mitchell, M. (2024, September). Adapting to the new normal: Understanding the impact of COVID-19 on technology usage and human behaviour. Entertainment Computing, 51, 100726. https://doi.org/10.1016/j.entcom.2024.100726
- Pypi. (n.d. -a). requests 2.32.3. Pypi. https://pypi.org/project/requests/
- Pypi. (n.d. -b). beautifulsoup4 4.13.4. Pypi. https://pypi.org/project/beautifulsoup4/
- Pypi. (n.d. -c). selenium 4.33.0. Pypi. https://pypi.org/project/selenium/
- Pypi. (n.d. -d). schedule 1.2.2. Pypi. https://pypi.org/project/schedule/
- Pypi. (n.d. -e). mysql-connector-python 9.3.0. Pypi. https://pypi.org/project/mysql-connector-python/
- Python Documentation. (n.d. -a). csv – CSV File Reading and Writing. Python. https://docs.python.org/3/library/csv.html
- Python Documentation. (n.d. -b). Unicode HOWTO. Python. https://docs.python.org/3/howto/unicode.html
- Raneri, P., Montag, C., Rozgonjuk, D., Satel, J., & Pontes, H. (2022, June). The role of microtransactions in Internet Gaming Disorder and Gambling Disorder: A preregistered systematic review. Addictive Behaviors Reports. 15, 100415. https://doi.org/10.1016/j.abrep.2022.100415
- Requests Documentation. (n.d.). Quickstartn. Requests Documentation. https://requests.readthedocs.io/en/latest/user/quickstart/
- Samuli, K. (2022, January 19th). Pricing economics of video games: a panel data study on the effects of versioning on revenue. University of Oulu Repository. https://urn.fi/URN:NBN:fi:oulu-202201191079
- Selenium. (n.d.). Locating Elements. Selenium. https://selenium-python.readthedocs.io/locating-elements.html
- ScrapeOps. (n.d.). How To Scroll Infinite Pages in Python. ScrapeOps. https://scrapeops.io/python-web-scraping-playbook/python-scroll-infinite-pages/
- Smith, Vincent. (2019). Go Web Scraping Quick Start Guide (1st Edition). Packt Publishing.

- Steam. (n.d.). Portal 2. Steam. https://store.steampowered.com/app/620/Portal_2/
- Steamworks. (n.d.). Steam Direct (Joining The Steamworks Distribution Program). Steam. https://partner.steamgames.com/steamdirect
- Tedboy. (n.d.). Beautiful Soup Documentation. Github. https://tedboy.github.io/bs4_doc/
- United Kingdom. (n.d.). Data protection (The UK's data protection legislation). United Kingdom. https://www.gov.uk/data-protection
- UKCS. (n.d.). UK copyright law: An introduction (The Copyright, Designs and Patents Act 1988). The UK Copyright Service. https://copyrightservice.co.uk/copyright/uk_law_summary
- Brown, M. (2012, February 9th). Tim Schafer persuades fans to finance next adventure game. Wired. Archive: https://web.archive.org/web/20120212003117/http://www.wired.co.uk/news/archive/2012-02/09/double-fine-kickstarter
- Xiao, G. (2021). BAD BOTS: REGULATING THE SCRAPING OF PUBLIC PERSONAL INFORMATION. Harvard Journal of Law & Technology, 34 (2), 701.
- Zhao, B. (2017). Web Scraping. Encyclopedia of Big Data. https://link.springer.com/rwe/10.1007/978-3-319-32001-4_483-1 ALT: https://www.researchgate.net/profile/Bo-Zhao-3/publication/317177787_Web_Scraping/links/5c293f85a6fdccfc7073192f/Web-Scraping.pdf
-